When Does LLM Orchestration Pay? GEAR: Graph-Controlled Execution with Adaptive Routing under Inference Budgets

Chen Wang[a,b], Jiangyao Bai[a,b], Zhaoyun Ding[a,b], Yijia Fu[a,b] and Lailong Luo[a,b,*]

[a]College of Systems Engineering, National University of Defense Technology, Changsha, 410073, China
[b]National Key Laboratory of Information Systems Engineering, National University of Defense Technology, Changsha, China

ARTICLE INFO

Keywords:
Large language models
Model routing
Execution graphs
Inference budgets
Distribution shift
Contextual bandits

ABSTRACT

Large language models (LLMs) increasingly serve as reasoning engines in knowledge-based systems, yet deploying them under budget constraints couples two critical decisions: model tier selection and execution topology. Existing routers allocate models only for flat single-call invocations, whereas fixed multi-agent frameworks incur substantial orchestration overhead on every incoming query. This paper proposes GEAR (Graph-Controlled Execution with Adaptive Routing), a principled framework that leverages task-structure knowledge to dynamically govern execution topology, reasoning depth, and model tier allocation under an inference budget. GEAR coordinates hierarchical step-level routing with closed-loop Lagrangian budget pacing and an uncertainty-guided outer escalation mechanism. Comprehensive empirical evaluations and deployment replays demonstrate that task-structure constraints substantially curtail token consumption relative to static orchestration without compromising reasoning accuracy. Furthermore, selective outer escalation consistently outperforms static execution policies by reserving multi-step graph construction exclusively for queries diagnosed with recoverable reasoning lapses. Consequently, GEAR establishes multi-agent orchestration as a targeted computational intervention rather than an unconditional default for sustainable compound LLM serving.

1. Introduction

Large language models (LLMs) have emerged as foundational reasoning engines in modern knowledge-based systems, powering applications ranging from multi-hop question answering and complex mathematical derivation to symbolic planning and structured retrieval [1–4]. In real-world enterprise and web-scale deployments, however, serving these models exposes profound trade-offs between inference expenditure and output reliability. Modern foundation models are organized into steeply tiered hierarchies, where proprietary frontier models (e.g., GPT-4 or Qwen-Max) incur order-of-magnitude higher monetary costs, token footprints, and processing latencies than lighter, faster counterparts [5, 6]. At the same time, the incoming workload of knowledge-intensive applications is inherently heterogeneous: while a subset of challenging queries demands exhaustive multi-step reasoning or high-capacity models, a substantial majority of routine inquiries can be accurately resolved by lightweight models via direct inference. The central operational challenge for sustainable knowledge-based systems is therefore not unconstrained quality maximization or retrospective cost accounting, but rather maintaining high task accuracy under stringent, non-stationary inference budget constraints [7–9].

To balance performance against operational expenditure, two distinct lines of research have developed in parallel. The first paradigm focuses on query-level model routing and static cascades [10–13]. These approaches assess input difficulty upfront or track predictive uncertainty [14–16] to route each query to an appropriate model tier in a single forward pass. While computationally frugal, their structural expressiveness is strictly bounded: they operate exclusively over flat, single-call invocations and are fundamentally incapable of decomposing problems or synthesizing multi-agent workflows. Conversely, the second paradigm pursues multi-agent orchestration and reasoning graph synthesis [17–20]. Frameworks employing iterative refinement, tree-structured search, or multi-agent debate [21–25] substantially elevate reasoning fidelity on complex tasks by breaking goals into sub-tasks and aggregating intermediate outputs. However, these orchestrators typically enforce monolithic, query-invariant execution topologies, incurring extensive token overhead and high invocation latencies on every single transaction, regardless of whether a simple format-matched call would have sufficed.

This dichotomy exposes a critical architectural gap in current LLM serving: query-level routers treat model selection as a standalone choice while freezing the execution topology to a single call, whereas compound multi-agent frameworks optimize reasoning topologies while treating orchestration as an unconditional, expensive default. In practical knowledge-based systems, model selection and execution topology are inherently coupled. Decomposing a query into sub-questions multiplies the number of downstream routing decisions, while the model tier assigned to an intermediate step dictates the marginal utility of subsequent critique, refinement, or expansion rounds. An optimal serving architecture must therefore jointly govern model tier selection and execution-graph topology under budget constraints, while retaining the explicit autonomy not to orchestrate when a direct response is sufficient.

*Corresponding author
sereneture@gmail.com (C. Wang); baijiangyao@nudt.edu.cn (J. Bai); zyding@nudt.edu.cn (Z. Ding); fuyijia20@nudt.edu.cn (Y. Fu); luolailong09@163.com (L. Luo)
ORCID(s): 0009-0003-2924-3399 (C. Wang)

Preprint not peer reviewed
This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

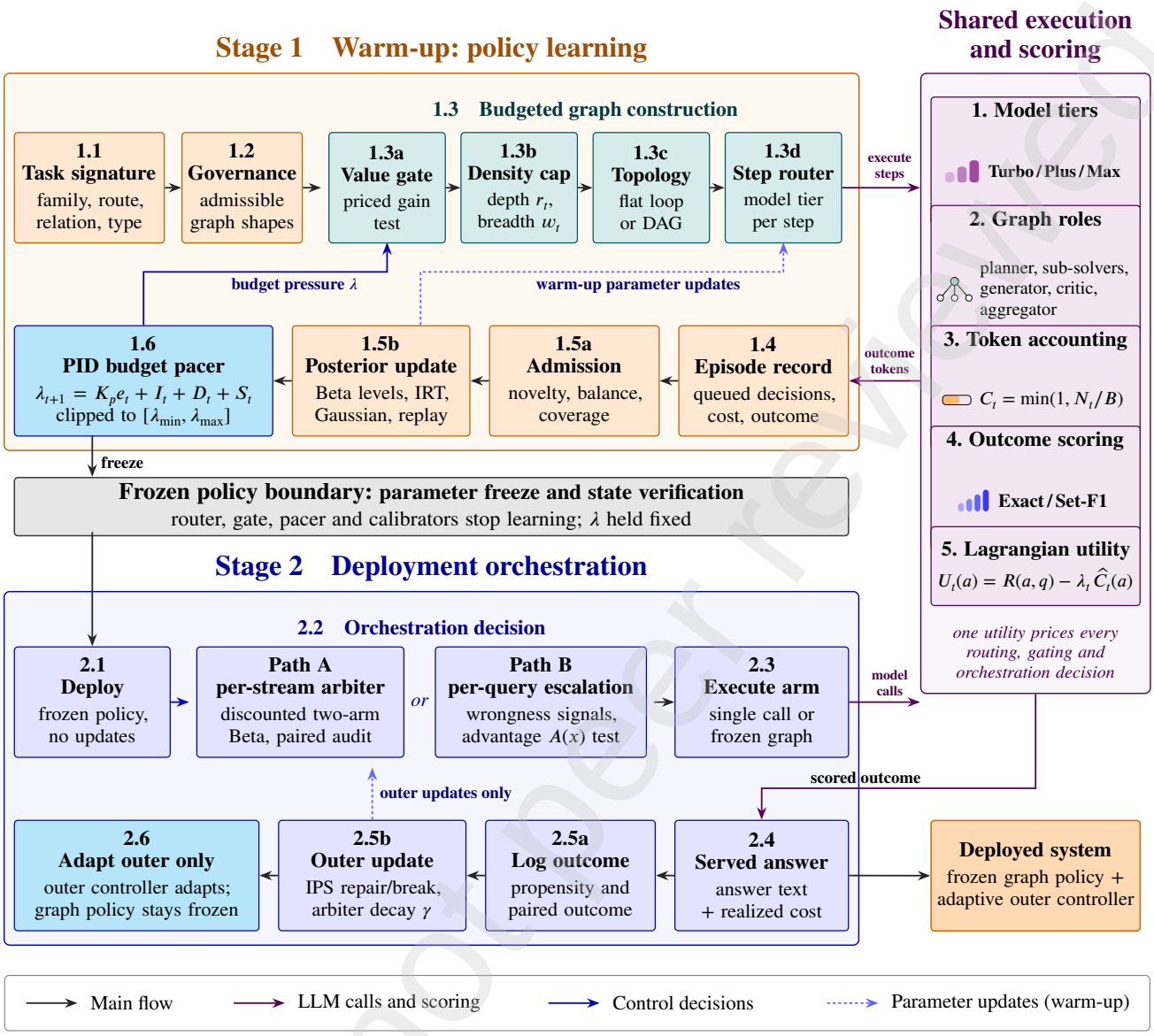


Figure 1: Technical route of GEAR. Stage 1 learns the execution-graph policy on a labeled warm-up stream: a task signature and a governance gate fix which graph shapes are admissible, the value gate, density cap and step router build one budgeted graph per query, and only admitted decisions update the posteriors and the PID pacer, whose multiplier λ_t prices every construction decision. The learned policy parameters are then frozen and verified. Stage 2 serves a deployment stream with the frozen policy and leaves free only the outer decision of whether to orchestrate, taken per stream by the workload arbiter or per query by post-execution escalation; the core graph policy no longer changes. The right column is shared by both stages, so every executed step is priced by the same Lagrangian utility.

To bridge this divide, this paper presents GEAR (Graph-Controlled Execution with Adaptive Routing), a principled serving framework illustrated in Fig. 1. GEAR formulates execution-graph construction as a constrained optimization problem, where the graph topology, expansion depth, and step-level model assignments are dynamically governed by a unified Lagrangian utility. Specifically, GEAR incorporates operational task-structure knowledge—such as answer cardinality, compositional dependencies, and mechanism compatibility—as deterministic governance rules that prune invalid topologies upfront without querying external knowledge bases. To ensure operational stability and eliminate continuous training overhead in production, GEAR follows a two-stage lifecycle: the core graph-policy parameters are learned on a compact warm-up stream and subsequently frozen, leaving only an outer orchestration decision active during deployment. This outer controller senses runtime uncertainty from an initial fast execution and selectively escalates to the frozen multi-step graph only when recoverable reasoning lapses are detected. By reserving multi-agent orchestration for queries that genuinely benefit from structural intervention, GEAR transforms agentic execution from an expensive default into a precision engineering tool.

The main contributions of this study are as follows:

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

1. This study formulates compound execution-graph construction as a budget-constrained utility maximization problem, where execution topology, expansion depth, and step-level model allocation are jointly regulated via closed-loop Lagrangian budget pacing.
2. Deterministic task-structure governance rules are introduced to prune inadmissible and degenerate graph configurations upfront, ensuring execution safety without querying external knowledge bases.
3. A two-stage serving architecture is developed to decouple warm-up policy calibration from prospective serving, employing an uncertainty-guided outer controller to escalate queries only upon detecting recoverable reasoning lapses.
4. Comprehensive empirical evaluations and deployment replays systematically characterize the operational boundaries of compound LLM systems across diverse reasoning benchmarks, establishing quantitative criteria for when structured graph orchestration justifies its computational cost over direct inference.

The remainder of the paper is organized as follows. Section 2 reviews related work. Sections 3 and 4 formalize the constrained optimization problem and detail the GEAR architecture. Section 5 presents our multi-tiered empirical evaluation, including locked benchmark replications, baseline comparisons, out-of-distribution probes, and deployment replays. Section 6 discusses operational principles and deployment implications, and Section 7 concludes.

2. Related Work

Four complementary research thrusts inform budget-constrained execution-graph control: cost-effective model serving, learned query routing and cascades, structured query decomposition and agentic workflows, and constrained feedback control. Each paradigm addresses a distinct tier within the LLM serving hierarchy (Table 1). In contrast to prior methodologies that treat model selection and execution topology as orthogonal, GEAR dynamically governs the execution graph itself, while retaining the explicit decision-theoretic autonomy to bypass multi-step orchestration when a direct response suffices.

2.1. Cost-Effective LLM Inference and Serving

System-level optimizations such as vLLM [26] and FlashAttention [27] raise serving throughput, while compression techniques such as AWQ [28] and GPTQ [29] reduce the cost of executing one model. Deployment and retraining studies consider service constraints and lifecycle costs [8, 9]. These methods are complementary to execution-graph control: they change the cost of an action, whereas the present problem is which actions and topology to execute.

2.2. Learned LLM Routing and Cascades

FrugalGPT [7] arranges tiers in a sequential cascade with answer scoring and fallback. RouteLLM [11], Hybrid LLM [12], and benchmark-based routing [10] learn query-level model selection from preference data or observed quality gaps, and RouterBench [32] standardizes their comparison. Online variants use bandits or primal–dual control under changing budgets [13, 33]. Uncertainty-aware routers use conformal sets or semantic entropy [15, 16], while X-Router separates knowledge from reasoning demand and failure-mode routing conditions the intervention on the anticipated error [14, 34]. These methods normally choose among single calls once per query. GEAR instead controls graph shape, depth, and per-step tiers, and its escalation controller can decide after observing the first call. We evaluate a learned query-level router across out-of-distribution boundary probes and comparative baseline benchmarks.

Table 1
Levels of control in budget-aware LLM serving. Each line of work fixes what the level above it decides. GEAR's decision variable is the execution graph, and its outer controllers decide whether to build one at all.

Level	Decision variable	Representative work
Execution graph	**Shape, depth, per-step tier**	**GEAR**
Workload	**Whether to orchestrate**	**GEAR (outer)**
Kernel/serving	Memory, attention schedule	[26, 27]
Weights	Numerical precision	[28, 29]
Model choice	One tier per query	[7, 11, 12]
Model choice	One tier, uncertainty-gated	[14, 15]
Reasoning shape	Fixed prompt template	[17, 18, 30]
Reasoning shape	Fixed agent roles	[19, 20, 31]
Workflow search	Offline graph / supernet search	[24, 25]

2.3. Query Decomposition and Agentic Workflows

Chain-of-thought [30], Least-to-Most [17], Decomposed Prompting [18], and Self-Ask [35] impose or generate reasoning structure. Tree search and reasoning–action interleaving broaden that structure [21, 22]. Knowledge-based variants use logic-tree retrieval, diversified generation, symbolic plans, or heuristic guidance [2, 4, 36, 37]. Refinement, self-consistency, test-time scaling, and backward verification spend additional inference within a template [3, 38–40]. Multi-agent systems similarly assign coordinated roles for construction, aggregation, repair, question generation, or replanning [19, 20, 23, 31, 41, 42]. Recent efforts automate workflow topology design: AFlow [24] applies Monte Carlo Tree Search over code-represented workflows to optimize static execution graphs, whereas MaAS [25] optimizes an agentic supernet to dynamically sample query-dependent agent structures. Both paradigms rely on extensive offline exploration, heavy simulation loops, or supernet pre-training. In contrast, GEAR targets online budget-constrained serving with minimal pre-deployment training overhead: instead of performing expensive structural MCTS or training continuous supernets, GEAR couples deterministic structural governance (enforcing admissibility rules upfront from query signatures) with lightweight empirical Bayes routing [43, 44] on a compact warm-up stream, and freezes this core policy before deployment. Outer post-execution escalation then

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

reserves multi-step graph execution strictly for queries where a single call exhibits detectable failure risks.

2.4. Feedback Control and Budget Pacing

Budget pacing by feedback control is established in real-time advertising [45]; PID Lagrangians have also been studied in constrained reinforcement learning [46]. Related formulations include bandits with resources [47], contextual Thompson sampling [48], discounted posteriors under non-stationarity [49], and budgeted federated allocation [50]. Primal–dual updates have recently been applied to LLM routing [33]. In summary, existing literature predominantly optimizes model selection over flat single-call invocations, executes static reasoning templates, or relies on computationally prohibitive offline searches to derive rigid agent graphs. In contrast, GEAR formalizes graph topology, expansion depth, and step-level tier allocation as lightweight, budgeted online decisions governed by deterministic task-structure knowledge, while establishing orchestration itself as a selective, sensor-guided post-execution intervention.

3. Problem Formulation

Let $\mathcal{M} = \{M_1, \dots, M_K\}$ be a portfolio of K heterogeneous LLMs ordered by cost (in our deployment, the Qwen-Turbo, Qwen-Plus, and Qwen-Max tiers). The operator receives a non-stationary stream of queries $q_1, q_2, \dots, q_T$. For each query q_t, the system commits to a compound execution action a_t that jointly resolves three tightly coupled operational decisions: (i) a structural decomposition indicator $D_t \in \{0, 1\}$ deciding whether the query is answered via an iterative flat trajectory ($D_t = 0$) or decomposed into a directed dependency graph of sub-questions ($D_t = 1$); (ii) a routing vector $\mathbf{m}_t$ assigning a specific model tier $M \in \mathcal{M}$ to each constituent node of the selected topology (encompassing planner, sub-question solvers, and aggregator under $D_t = 1$, or generator, critic, and aggregator across refinement rounds under $D_t = 0$); and (iii) an adaptive temporal stopping policy governing whether to terminate execution, continue at the current model tier, or escalate to a higher-capacity tier following each reasoning round. These decisions are coupled: topological decomposition multiplicatively expands downstream routing states, whereas intermediate model tier assignments directly dictate the expected marginal utility of subsequent critique and refinement rounds.

Executing a_t consumes $N(a_t, q_t)$ controller-accounted tokens and yields quality $R(a_t, q_t) \in [0, 1]$ (Exact Match for scalar answers, Set-F1 for set-valued answers). Relative to per-episode token budget B (set to 28,000 tokens), the controller uses

$$C_t = \min\{1, N(a_t, q_t)/B\}. \tag{1}$$

This internal, provider-independent token fraction is distinct from the USD expenditure reported in experimental tables using historical list prices. Here, the unit-interval clamp preserves Lagrangian dual stability; across all empirical executions, the maximum observed episode token consumption is $27,242 < B$, meaning the clamp is never binding and does not mask realized expenditure. With target token fraction $\rho \in (0, 1)$, the objective is

$$\max_{\{a_t\}} \quad \sum_{t=1}^{T} \mathbb{E}\left[R(a_t, q_t)\right], \tag{2}$$

$$\text{subject to} \quad \frac{1}{T}\sum_{t=1}^{T} C_t \le \rho. \tag{3}$$

Lagrangian relaxation with dual variable $\lambda \ge 0$ yields the per-step utility

$$U_t(a_t) = R(a_t, q_t) - \lambda_t \cdot \widehat{C}_t(a_t), \tag{4}$$

which every decision in GEAR maximizes. Maximizing Eq. (4) in non-stationary serving environments presents three fundamental decision-theoretic challenges: (i) *Partial counterfactual feedback*: quality and resource expenditure are observed solely for the executed path, requiring unchosen topologies to be statistically inferred; (ii) *Dynamic budget non-stationarity*: non-uniform workload arrival precludes static dual pricing, requiring closed-loop adaptation to enforce global budget compliance; and (iii) *Compositional experience transfer*: historical execution traces generalize reliably only among queries sharing structural signatures, an inductive bias absent from raw token-utility metrics. Section 4 articulates the core architectural mechanisms developed to resolve these challenges.

4. The GEAR Framework

The architectural design of GEAR is guided by a unified optimization and deployment principle: govern all topological and model selection decisions via a shared Lagrangian utility, calibrate and freeze this core structural policy over a compact warm-up stream to guarantee execution invariance, and reserve adaptive meta-control solely for the outer deployment decision of whether to invoke multi-step orchestration. As schematized at the macro lifecycle level in Fig. 1 and organized across operational tiers in Table 2, this design decouples invariant graph generation from runtime workload adaptation, ensuring both statistical predictability and execution efficiency. At the query level, the operational workflow and task-structure-governed execution pathways are detailed in Fig. 2.

GEAR is structured into three foundational structural components complemented by two deployment-stage meta-controllers. The foundational tier comprises: (i) an online neural-Bayesian step router governing node-level model allocation, (ii) a value-of-decomposition gate coupled with PID Lagrangian budget pacing, and (iii) deterministic task-structure governance. These mechanisms span four operational levels (summarized in the upper partition of Table 2) that undergo rapid parameter calibration during an initial warm-up phase and remain strictly frozen during prospective serving. In contrast, the deployment tier introduces two adaptive meta-controllers (the lower partition of Table 2):

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Table 2
Structural organization of the GEAR framework: four internal operational levels (upper partition) undergo calibration and are frozen post-warmup; two deployment meta-controllers (lower partition) govern runtime serving adaptation. Learned levels optimize the unified Lagrangian utility in Eq. (4).

Level	Decision	State, and when it moves
Step router	Model tier for each executed step	Contextual Gaussian heads, IRT, neural ensemble, four Beta levels; warm-up only
Decomposition gate	Flat or decomposed execution	Value-of-decomposition estimates and density cap; warm-up only
Budget pacer	Cost pressure λ_t entering every score	PID state over the realized budget ratio; warm-up only
Governance	Which mechanisms are safe to enable	None; deterministic rules read off the task signature
Workload arbiter	One call or orchestration, per stream	Discounted two-arm posterior; adapts during deployment
Escalation rule	Commit or orchestrate, after the cheap call	Wrongness detector and discounted repair/break rates; adapts during deployment

a stream-level workload arbiter and an instance-level post-execution escalation policy. While all decision mechanisms maximize the unified Lagrangian utility in Eq. (4), they operate at distinct spatial and temporal granularities across the query lifecycle.

4.1. Online Neural-Bayesian Step Router

At each discrete execution step—encompassing state construction, candidate generation, critique, aggregation, sub-question planning, and temporal termination gates—model tier allocation is governed by a unified online neural-Bayesian routing mechanism.

4.1.1. Hybrid quality estimation

For a step with hashed query and dense context features $\boldsymbol{x}_t$ (task type, calibrated difficulty, confidence, budget-used fraction, current λ_t, round, phase, and role), each candidate action a has a diagonal Bayesian linear head. Sampling its coefficients gives a contextual Thompson estimate [48]

$$\tilde{q}^{\mathrm{GTS}}_{t,a} = \sigma\left(\boldsymbol{x}_t^{\top}\tilde{\theta}_a\right), \quad \tilde{\theta}_{a,j} \sim \mathcal{N}\left(\frac{b_{a,j}}{P_{a,j}}, \frac{\gamma^2}{P_{a,j}}\right), \tag{5}$$

where $P_{a,j}$ and $b_{a,j}$ are updated only for the executed action. The IRT head [51] separates an action's ability from an item's difficulty and predicts $q^{\mathrm{IRT}}_{t,a} = \sigma(\eta_a + s_{a,\mathrm{task}} + p_{a,\mathrm{phase}} - d(\boldsymbol{x}_t))$. A bootstrapped multilayer-perceptron ensemble supplies $\tilde{q}^{\mathrm{NN}}_{t,a}$, sampled from one ensemble member with a disagreement bonus. In parallel, four Beta posteriors are maintained for the specific context, task type, difficulty bucket, and global action statistics. Pooling them trades the low bias of the specific cell against the low variance of the coarse one, which is the classical shrinkage trade-off [43, 44]. Their sample is

$$\tilde{q}^{\mathrm{H}}_{t,a} = \frac{\sum_{\ell\in\{\mathrm{spec,task,bucket,global}\}} \omega_{\ell,t}\tilde{z}_{\ell,a}}{\sum_{\ell}\omega_{\ell,t}}, \qquad \tilde{z}_{\ell,a} \sim \mathrm{Beta}(\alpha_{\ell,a}, \beta_{\ell,a}), \tag{6}$$

with static fallback relative weights $1.0/0.3/0.3/0.1$ for specific/task/bucket/global. The default configuration enables auto-pooling: once at least three eligible specific cells are available for an action, it estimates a Beta–binomial concentration $\hat{\kappa}_a$, assigns the specific level weight $n_{\mathrm{spec}}/(n_{\mathrm{spec}}+\hat{\kappa}_a)$, and rescales the three coarser weights to the remaining mass while preserving their $0.3:0.3:0.1$ ratio. Before that estimate is available, it falls back to the static weights. The final sampled quality and action are

$$\tilde{Q}_{t,a} = \frac{w_b\tilde{q}^{\mathrm{GTS}}_{t,a} + w_i q^{\mathrm{IRT}}_{t,a} + w_n\tilde{q}^{\mathrm{NN}}_{t,a} + w_h\tilde{q}^{\mathrm{H}}_{t,a}}{w_b + w_i + w_n + w_h}, \tag{7}$$

$$a^*_t = \arg\max_{a\in\mathcal{A}_t}\left[\tilde{Q}_{t,a} - \lambda_t\hat{C}(a)\right]. \tag{8}$$

Only a^*_t is observed. Thus "partial feedback" means that no reward is imputed for unexecuted models or flow actions. To prevent a query's label from changing later decisions on that same query, executed decisions are queued until the episode has produced its final answer. The four hierarchy levels, contextual head, IRT parameters, and neural replay for an admitted queued action are then updated from the completed episode outcome and that action's attributed cost.

4.1.2. Data-centric sample admission and active exploration support

To prevent redundant updates across identical queries and maintain statistical balance, learning instances are indexed by canonical query representations rather than transient execution keys. For a labeled warm-up episode i, let h_i denote this canonical index, f_i its task family, y_i the observed task outcome, and $\hat{q}_{i,a}$ the logged posterior mean for an executed action. A queued routing decision is admitted to the training buffer when the question is novel, the cumulative task-family proportion satisfies the balance cap, and at least one of three informativeness criteria holds:

$$\begin{aligned}\mathcal{A}_{i,a} = \mathbb{I}[h_i \notin \mathcal{H}]\,\mathbb{I}[\mathrm{share}(f_i) \le \pi_{\max}] \\ \times \mathbb{I}[n_{\mathrm{cell}} < n_{\min} \ \vee\ |y_i - \hat{q}_{i,a}| \ge \tau_e \\ \vee\ \mathrm{range}(\hat{\mathbf{q}}_i) \ge \tau_c].\end{aligned} \tag{9}$$

The first two factors prevent query repetitions from inflating posterior precision and guard against task-distribution skew. The third prioritizes under-covered cells, substantial prediction errors, and decisions where candidate actions exhibited high posterior divergence. Only verified ground-truth warm-up episodes update the IRT parameters, Bayesian linear heads, or neural replay; heuristic proxies and all evaluation

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

instances are strictly excluded. The admission mechanism retains compact structural indices and occupancy statistics rather than raw text, and is frozen alongside the routing policy.

Passive sample admission alone cannot identify an action that the serving policy never executes. GEAR therefore pairs Eq. (9) with targeted active exploration constraints for selected (phase, task, action) tuples during warm-up. While an exploration quota remains unmet, the router prioritizes the most under-covered feasible action; this mechanism is deactivated during frozen evaluation. Specifically, the framework enforces a minimum exploration threshold of 24 observations for the mid-tier solver on mathematical tasks. This represents an empirical coverage requirement rather than synthetic feedback: every parameter update is computed from an actual execution and verified label.

4.1.3. Temporal control

The same router governs the iterative refinement loop. After round r, it chooses among three actions: Continue, Upgrade (which escalates the generator to a strictly stronger model tier), or Stop. The decision context incorporates answer-stability and answer-quality indicators. The following inequality characterizes the marginal-utility learning target:

$$\mathbb{E}\left[\Delta R^{(r+1)}\right] - \lambda_t \cdot \mathbb{E}\left[\Delta C^{(r+1)}\right] < 0. \tag{10}$$

Rather than relying on ad-hoc heuristic thresholds, the temporal controller dynamically selects candidate actions under partial feedback, wherein Eq. (10) formalizes the decision-theoretic condition under which negative expected marginal utility dictates immediate termination (Stop). Two complementary stopping mechanisms are employed. Prior to topology expansion, the initial single-call stage produces a candidate answer alongside self-reported confidence, an evidence-completeness assessment, and estimated answer cardinality. An early-exit mechanism accepts this answer directly only when it is well-formed, complete, and accompanied by calibrated confidence exceeding the acceptance threshold, provided that the reported cardinality aligns with the task structure estimate; multi-hop routes, set-valued answers, and multi-entity targets are protected from premature exit. Raw self-reported confidence is not comparable across tiers, so it is mapped through a Platt scaler [52], the standard remedy for uncalibrated neural scores [53]. Its parameterization is initialized to the identity mapping until sufficient calibration observations accumulate. Within refinement, an intermediate solver gate can bypass critic/aggregator calls when the candidate answer satisfies strict confidence and formatting checks. All routed flow actions, including Stop, receive delayed episode-level utility once the final response is established.

4.2. Value-of-Decomposition Gate and Lagrangian Pacing

4.2.1. Value of decomposition

The gate maintains, per (task type, difficulty bucket) cell, exponential moving averages of quality and cost under both execution modes, updated with clipped inverse-propensity weighting under its stochastic policy. When decomposition is declined and the flat path executes, that observed flat outcome updates the flat arm; it is not treated as feedback for the unexecuted decomposed arm. Decomposition is accepted when

$$\bar{R}_t(1) - \bar{R}_t(0) \;\geq\; \delta_{\min} + \lambda_t\left(\bar{C}_t(1) - \bar{C}_t(0)\right) - b_{\text{explore}} - b_{\text{ram}}, \tag{11}$$

where $\delta_{\min}$ is a minimum-gain margin, b_{explore} is a cold-start exploration bonus that decays with cell sample count, and b_{ram} is the optional memory bias from task-adaptive governance (zero in the mainline).

4.2.2. PID Lagrangian pacing

After each episode, the normalized token expenditure is computed as $c_t = [N_t/B]_0^1$, an exponential moving average is updated via $\bar{c}_t = (1-\alpha)\bar{c}_{t-1} + \alpha c_t$, and the tracking error is defined as $e_t = \bar{c}_t - \rho$. Its PID state and multiplier are

$$\begin{aligned} I_t &= \max\{0, I_{t-1} + K_i e_t\}, \\ D_t &= K_d \max\{0, e_t - e_{t-1}\}, \\ S_t &= K_s \max\{0, c_t - \tau_s\}, \\ \lambda_{t+1} &= [K_p e_t + I_t + D_t + S_t]_{\lambda_{\min}}^{\lambda_{\max}}. \end{aligned} \tag{12}$$

Thus λ is assigned from the current PID/Lyapunov terms rather than incremented by them. The rectified integral does not accumulate under-budget credit, the positive derivative reacts only to worsening error (and is zero on the first observation), and the spike term activates only above the configured threshold τ_s, not at a fixed multiple of ρ. The resulting λ_t is consumed by *every* step-routing and decomposition decision, so budget pressure uniformly reshapes the execution graph: shallower refinement, cheaper tiers, and stingier decomposition as λ_t rises.

4.2.3. Structure-aware density cap

While the value gate in Eq. (11) governs the discrete structural decision of whether to decompose a query, it does not regulate the structural density or expansion breadth of the resulting execution graph. Let bucket $\beta_t \in \{0, \ldots, N_\beta - 1\}$ be the calibrated difficulty bucket (0 easiest, N_β the number of buckets), and let $\hat{m}_t$ be the structure policy's estimated answer cardinality. The cap maps β_t to monotone depth and breadth schedules. Its breadth floor is computed from configured minima, $\hat{m}_t$, answer type, task family, and route; cardinality therefore protects a broad answer even when its scalar difficulty is underestimated. Budget pressure tightens only structures that are not protected (unless explicitly configured otherwise). With $f_t = \max\{w_{\min}, \hat{m}_t, f_{\text{type}}, f_{\text{route}}\}$ and $z_t = \mathbb{1}[\lambda_t/\lambda_{\max} \geq \tau \wedge \neg\text{protected}(\boldsymbol{k}_q, \hat{m}_t)]$:

$$\begin{aligned} r_t &= \left[\, r_{\max}(\beta_t) - z_t \,\right]_{r_{\min}}^{R}, \\ w_t &= \max\left(f_t, \left[\, w_{\max}(\beta_t) - z_t \,\right]_{w_{\min}}^{W}\right), \end{aligned} \tag{13}$$

where R, W are global maxima and $[\cdot]_a^b$ clamps to $[a, b]$. The temporal loop runs at most r_t rounds and the planner emits at

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

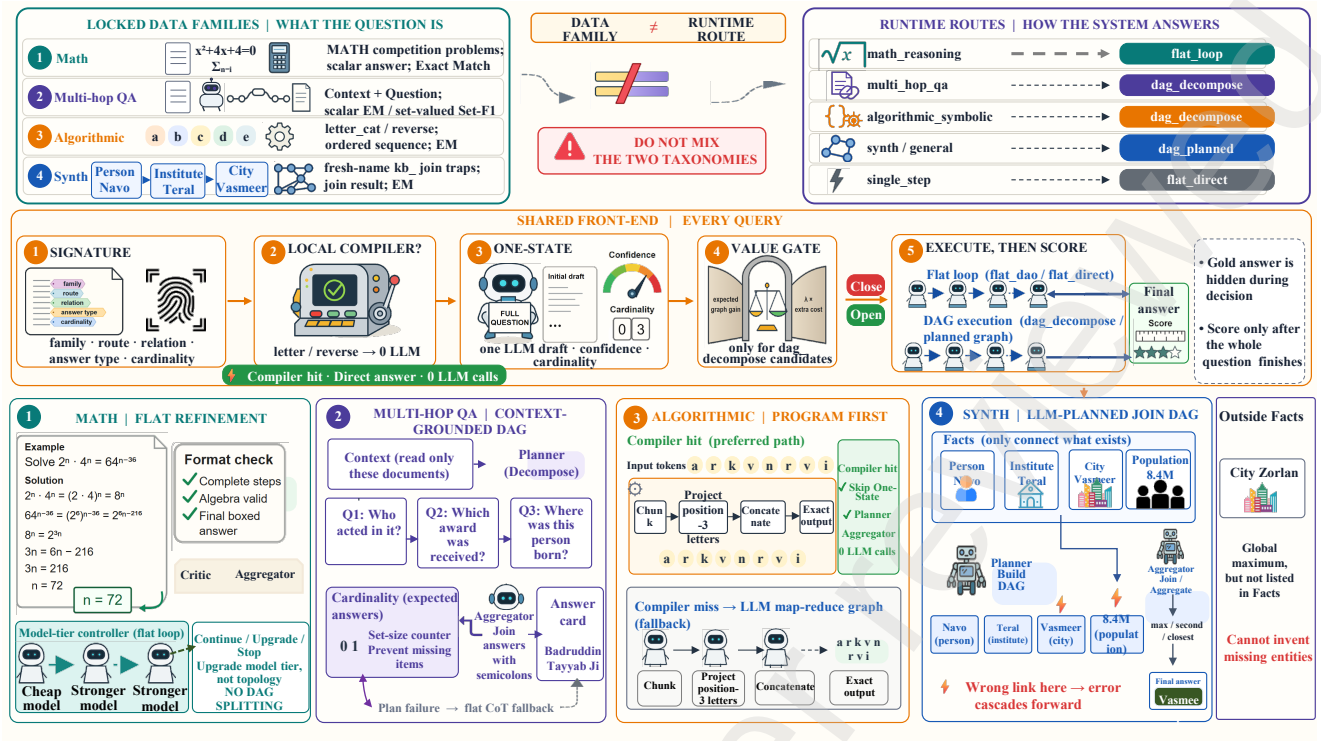


Figure 2: Task-structure-governed query execution pipeline and specialized runtime execution pathways in GEAR. Top: strict conceptual separation between problem categories (*Locked Data Families*) and execution strategies (*Runtime Routes*). Middle: shared five-stage front-end traversed by every incoming query, coordinating structural signature extraction, local compiler shortcuts (achieving zero LLM invocations), one-state drafting, value-of-decomposition gating, and delayed episode scoring. Bottom: four specialized runtime execution pathways tailored to intrinsic task structures, illustrating flat iterative refinement with model-tier upgrading for mathematical reasoning, context-grounded DAG decomposition for multi-hop QA, compiler-first execution with LLM map-reduce fallback for algorithmic tasks, and fact-constrained DAG planning for synthetic relational joins.

most w_t sub-questions. Easy-stop and the cap interact through the same cardinality estimate: protected broad/set structures cannot terminate at One-State and cannot be narrowed below their breadth floor. Both controls are optional and evaluated separately from the mainline.

4.3. Task-Adaptive Governance

4.3.1. Task Structure Policy and Taxonomy Decoupling

As illustrated in Fig. 2 (top), a central governing principle of GEAR is the strict conceptual decoupling between data families (*what the question is*) and runtime routes (*how the system answers*). Data families classify incoming queries into four benchmark domains: mathematical competition problems, context-grounded multi-hop reasoning, algorithmic sequence transformations, and synthetic relational knowledge joins. In contrast, runtime routes represent architectural execution policies: `flat_loop` for iterative refinement without topology expansion, `dag_decompose` for parallel sub-question decomposition, `dag_planned` for dynamic LLM-synthesized dependency graphs, and `flat_direct` for single-call queries. Conflating these two taxonomies leads to rigid, suboptimal serving; GEAR enforces deterministic structural mapping while allowing downstream controllers to adaptively select execution modalities.

A lightweight deterministic classifier maps each incoming query to a structural signature

$$\boldsymbol{k}_q = \big(\text{family}, \text{route}, \text{signature}, \text{format}\big), \tag{14}$$

e.g., (Multi-hop, Decompose, acted+award+born, Set) for a CommaQA query or (Single-step, Flat, None, Scalar) for a MATH query. The signature provides the routing context, selects which queries are decomposition candidates, and dictates the strict output format (semicolon-separated sets versus scalar values) used for parsing and online reward.

4.3.2. Shared Five-Stage Front-End Pipeline

As depicted in Fig. 2 (middle), every incoming query traverses a unified five-stage front-end before committing computational resources:

1. *Signature*: Parses query tokens into structural attributes $\boldsymbol{k}_q$, identifying task family, candidate route, entity relations, output type, and expected answer cardinality.
2. *Local Compiler?*: Evaluates whether deterministic programmatic rules apply (e.g., character projection or sequence reversal in algorithmic tasks). Upon a compiler hit, closed-form symbolic code directly outputs the exact solution with zero LLM calls, bypassing downstream models entirely.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

3. *One-State*: Generates an initial candidate solution via a single model call, evaluating self-reported confidence and estimating answer cardinality to detect whether multi-entity coverage is required.
4. *Value Gate*: For queries designated as `dag_decompose` candidates, the gate evaluates Eq. (11) to determine whether expected graph reasoning gains justify the Lagrangian-priced token cost. Queries branch into the Open path (proceeding to multi-step DAG execution) or the Close path (retaining flat iterative refinement).
5. *Execute, Then Score*: Dispatches execution to either the iterative flat loop (`flat_loop` / `flat_direct`) or dependency DAG execution (`dag_decompose` / `dag_planned`). Gold answers remain strictly hidden during execution, and performance rewards are assigned only upon full episode termination.

4.3.3. Specialized Task Execution Pathways

At the execution stage (Fig. 2, bottom), queries follow four specialized, structure-governed pathways:

1. **Math reasoning** (`flat_loop`): Enforces a strict "No DAG Splitting" governance constraint. Because mathematical derivations and algebraic transformations suffer from contextual fragmentation when partitioned into disconnected sub-questions, GEAR routes mathematical queries through iterative flat refinement. A model-tier controller escalates model capacity (Cheap → Stronger) rather than topological complexity, while an intermediate critic verifies algebraic correctness and final boxed formatting.
2. **Context-grounded multi-hop QA** (`dag_decompose`): Employs a planner to decompose complex questions into independent sub-queries (Q_1, Q_2, Q_3). Dedicated subsolvers read evidence strictly from provided context documents. An explicit set-size counter regulates answer cardinality to prevent omitted items before semicolon-separated aggregation, with an automatic fallback to flat Chain-of-Thought upon plan invalidity.
3. **Algorithmic reasoning** (`dag_decompose`): Implements a compiler-first execution paradigm. Deterministic operations (e.g., projecting position-k letters across token chunks) achieve immediate exact output with zero LLM invocations. Only upon a compiler miss does the system fall back to an LLM map-reduce execution graph.
4. **Synthetic relational joins** (`dag_planned`): Deploys an LLM planner to construct multi-hop join graphs across documented entities. To prevent hallucination traps where models invent non-existent entities (such as unlisted global maxima outside the context), the aggregator strictly enforces closed-world factual constraints, preventing error cascading across sequential links.

4.3.4. Task-adaptive component control

Auxiliary components are not globally enabled. The governance layer first evaluates whether a mechanism is structurally compatible with the current execution graph; only compatible mechanisms may influence routing. Episodic routing memory is therefore kept outside the default GEAR configuration and used only as an illustrative test of this rule. When enabled, it is restricted to permissible compositional routes (such as linear decomposition); on single-step flat-reasoning routes, it remains inactive. This keeps the primary architecture focused on topology-aware online orchestration rather than on global experience retrieval.

4.3.5. Diagnostic conservative episodic routing memory

For completeness, the diagnostic memory stores tuples $\{q_i, \boldsymbol{k}_{q_i}, D_i, U_i\}$ of past structural decisions and their realized utilities. At query time it retrieves neighbors only within the same structural signature and may bias the decomposition gate only when the retrieved evidence is both similar and low-variance:

$$b_{\text{ram}} = w_{\text{ram}} \cdot \mu_{knn} \cdot \mathbb{1}[\mu_{knn} \geq \mu_{\min}], \tag{15}$$

where w_{ram} denotes the maximum bias magnitude, with abstention unless retrieved neighbor density, maximum similarity, and variance criteria are concurrently met. This auxiliary component is evaluated as an illustrative ablation rather than a primary contribution, demonstrating how governance prevents unvalidated secondary mechanisms from perturbing the execution topology unless their gains are consistent across task families.

4.3.6. Design simplifications

GEAR deliberately omits several mechanisms that a topology-aware orchestrator might plausibly include: per-subquestion verifier scores with Bayesian credit assignment to subsolvers, pruning of the decomposition plan's directed acyclic graph, hand-coded marginal-utility stopping heuristics, and episodic memory enabled by default. In our diagnostics none of them improved on global-outcome reward with memory-free governance, so the reported system keeps episode-level credit assignment and a single learned stopping policy. Easy-stop and the cardinality-aware density cap are likewise treated as optional controls and are evaluated as separate frozen ablations in Section 5.2 rather than folded into the mainline.

4.4. Workload-Level Arbitration

While the foundational structural components govern internal graph topology and step-level tier allocation (resolving *how* to orchestrate), the first deployment controller operates at the macro-workload level, determining whether an incoming query stream warrants multi-step orchestration over direct single-model serving (resolving *whether* to orchestrate). Because pre-execution query surface features fail to provide reliable discriminative separation within homogeneous workloads, the arbiter formalizes this decision as an online two-armed bandit between a format-matched baseline call and the frozen GEAR execution policy.

The arbiter maintains one discounted Beta posterior per macro-arm $a \in \{\text{SINGLE}, \text{ORCH}\}$, where SINGLE is one format-matched call to the mid tier and ORCH is the full

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

frozen GEAR policy. After observing a binary outcome z for arm a, the posterior updates with decay γ,

$$\alpha_a \leftarrow 1+\gamma(\alpha_a-1)+z, \qquad \beta_a \leftarrow 1+\gamma(\beta_a-1)+(1-z), \quad (16)$$

so old evidence fades and the posterior re-widens under drift. Each arm also carries a running mean cost $\bar{c}_a$. For every query the arbiter draws k paired samples $\theta_a \sim \mathrm{Beta}(\alpha_a, \beta_a)$ and compares Lagrangian utilities $u_a = \theta_a - \lambda \bar{c}_a$—the same quality-minus-priced-cost objective the step router optimizes—yielding a lead probability p_{lead} for the winning arm. The leader is served. With probability $p_{\text{audit}} = \max\big(p_{\min}, 2(1 - p_{\text{lead}})\big)$ the other arm is executed as well, and both outcomes update their posteriors; otherwise only the served arm's outcome is recorded. Audits are thus scheduled by posterior uncertainty: frequent while the arms are indistinguishable, decaying to the floor $p_{\min}$ once they separate, and resuming automatically when discounting re-widens the posteriors after a workload shift.

The arbiter operates outside the frozen graph policy. Step-level and query-level execution policies remain strictly frozen and verified during evaluation. The arbiter maintains only a low-dimensional parametric state (a scalar Beta posterior and running cost per arm), precluding memorization of evaluation items. Online adaptation is essential to its function: whether orchestration confers a net benefit is an intrinsic property of the deployment stream that cannot be determined at warm-up time. In our evaluation, the two competing macro-arms correspond to a format-matched mid-tier call and the frozen GEAR execution policy; this meta-decision is strictly decoupled from GEAR's internal routing heads. The governing utility cost corresponds to the normalized token fraction N/B, matching the Lagrangian formulation of Section 4.2; monetary expenditures in USD are reported separately and not incorporated into λ. Full-arm counterfactual trajectories may be logged for offline analysis, but online belief updates are restricted strictly to the served arm, augmented by the audited arm only when an audit was explicitly scheduled. We detail this scoped adaptation protocol in Section 5.5.

4.5. Per-Query Escalation as a Constrained Bandit

Complementing stream-level arbitration, the second deployment controller formalizes per-query escalation as an instance-level constrained contextual bandit that operates *post-execution*—subsequent to an initial format-matched single-model call. Rather than committing unconditionally to either flat serving or multi-agent orchestration, the escalation policy evaluates runtime diagnostic signals (token-level confidence distributions, consensus with an auxiliary lightweight scout model, and deterministic invariant checks) to detect potential execution failure. Multi-step graph orchestration is triggered selectively only when the estimated probability of error indicates that the expected utility of structural repair exceeds the incremental inference expenditure.

Objective. All of GEAR's spending decisions maximize expected quality subject to a budget, so each shares the Lagrangian reward

$$R(q, a) \;=\; \mathbb{1}[\text{answer correct}] \;-\; \lambda\, c(q, a), \qquad (17)$$

where a indexes the executed action, $c(q, a) = N(a, q)/B$ is the same token fraction used by the graph-level Lagrangian, and λ is the multiplier of that spending constraint rather than an arbitrary dollar-scaled tuning factor, instantiating the budget-constrained contextual bandit framework [47]. In turn, modulating λ_t according to tracking error provides a smoothed proportional-integral-derivative approximation to primal-dual subgradient updates $\lambda \leftarrow [\lambda + \eta(\bar{c} - \rho)]_+$, connecting the PID pacing controller in Section 4.2 directly to constrained dual descent. Under the strict frozen evaluation protocol, λ is fixed to establish a rigorous evaluation baseline, whereas dynamic pacing is evaluated under online streaming sweeps. In this setting, the escalation decision reduces to evaluating the contextual advantage of invoking the execution graph over committing to the preliminary single-call response:

$$\begin{aligned} A(x) &= \mathbb{E}\big[R(q, \text{ESC}) - R(q, \text{COMMIT}) \mid x\big] \\ &= \hat{p}\, r - (1 - \hat{p})\, b - \lambda\, c_{\text{gear}}, \end{aligned} \qquad (18)$$

Here $\hat{p}$ is the estimated probability that the single answer is wrong. The rates r and b, at which orchestration respectively repairs a wrong answer and breaks a right one, are held as discounted Beta posteriors, conditioned on the task signature and shrunk toward the stream-level rate.

Paired advantage. On an escalated query both arms are observed, so their within-query difference is $\Delta = (\mathbb{1}[\text{GEAR}] - \mathbb{1}[\text{SINGLE}]) - \lambda c_{\text{gear}}$. Pairing removes item difficulty from this contrast. We mean-center the two outcomes, as in group-relative advantage estimation [54], but do not variance-normalize: with two arms normalization would discard the cost magnitude required by Eq. 17.

Propensity scoring and selection debiasing. A deterministic greedy threshold $A(x) > 0$ lacks a closed-form action propensity, inducing severe observational selection bias: because escalation is driven by error detection ($\hat{p}$), naive transition statistics reflect *detectably* flawed queries rather than the general population distribution. To ensure rigorous counterfactual identification, we formalize the escalation policy as a smoothed stochastic mapping:

$$\pi(\text{ESC} \mid x) \;=\; (1 - \epsilon)\, \sigma\big(A(x)/\tau\big) \;+\; \tfrac{1}{2}\epsilon, \qquad (19)$$

and log the realized propensity. The mixing floor ϵ is no longer an ad hoc exploration term: it bounds the propensity below by $\epsilon/2$ and hence the inverse-propensity weights by $2/\epsilon$, which is what keeps the corrected estimator's variance finite. The detector itself is trained unweighted on every query, because the cheap arm runs regardless of the action and its outcome is therefore observed under no selection at all. Only the rate posteriors need correcting. They are updated with self-normalized inverse-propensity weights [55], rescaled by their running mean so that debiasing a rate does not also inflate

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Algorithm 1 GEAR warm-up and frozen-evaluation loop. Warm-up observations serve as training data, not evaluation samples. Parameter freezing establishes the evaluation boundary; strict frozen evaluation keeps the Lagrangian multiplier fixed as well. An optional adaptive pacing mode is evaluated separately in supplemental diagnostics.

Input: ordered warm-up stream $\mathcal{W}$, evaluation stream $\mathcal{E}$, token budget B, model/action sets
Initialize router, hierarchy, IRT, neural replay, sample selector, calibrators, cluster index, value gate, and Lagrangian state
for each $(q, y) \in \mathcal{W}$ **do**
    Infer task signature, decomposition eligibility, answer type, and cardinality; construct initial candidate signal
    if structure permits early exit and calibrated confidence accepts the candidate answer **then**
        Accept the candidate answer and record its observed quality and cost
    else
        Compute round and sub-question caps from difficulty, structure/cardinality floors, and λ
        if decomposition is structurally eligible and the value gate accepts **then**
            Route planner, executed subsolvers, and aggregator; observe final quality and cost
        else
            Initialize the flat answer from candidate response
            for each round up to the computed round cap **do**
                Route Continue / Upgrade / Stop; break on Stop
                Execute the routed flat generator/critic/aggregator steps and update the current answer
            end for
            Observe final flat quality and cost
        end if
    end if
    After final answer is fixed, apply admission filtering (novelty, family balance, coverage); update admitted queued actions from episode quality and cost
end for
Freeze all learned parameters and verify state integrity
Disable router, hierarchy, IRT, neural replay, calibrator, cluster, and value-gate updates; fix λ for strict evaluation
for each $(q, y) \in \mathcal{E}$ **do**
    Verify state invariance before inference; execute policy without updates
    Confirm post-inference state invariance; record evaluation metrics
end for

the pseudo-counts behind its posterior variance. Shadow collections of the unexecuted orchestration arm, when gathered for confirmatory analysis, are excluded from these updates unless the logged propensity shows that escalation actually ran.

4.6. Algorithms and Feedback Boundary

5. Experimental Evaluation

Our empirical evaluation investigates the performance of GEAR across diverse reasoning domains, budget constraints, and operational regimes, focusing on five central research questions: whether unified Lagrangian graph control attains superior cost-accuracy Pareto trade-offs relative to static flat orchestration (Section 5.2); how GEAR compares against representative serving baselines including high-tier proprietary models, learned query routers, and test-time majority voting (Section 5.3); under which resource bottlenecks graph orchestration yields net advantages (Section 5.4); how effectively post-execution uncertainty estimators and invariant auditors detect recoverable single-call failures (Section 5.6); and whether selective escalation outperforms fixed universal policies in end-to-end streaming deployment (Section 5.7).

Our experimental protocols distinguish two operating modes: (i) the *frozen policy evaluation*, where core routing parameters, IRT difficulties, and decomposition gates are trained on a labeled warm-up stream and strictly frozen to evaluate generalization invariance; and (ii) the *streaming deployment replay*, where queries arrive sequentially under token budgets, and outer controllers adaptively govern orchestration actions under causal feedback masking.

5.1. Experimental Setup

The locked replication suite comprises 200 labeled warm-up queries followed by 600 evaluation queries, balanced across four 150-query families: context-supplied multi-hop questions from CommaQA-E [56], MATH problems [57], algorithmic-symbolic tasks adapted from Decomposed Prompting [18], and a fresh synthetic namespace. Thus 150 of 600 evaluation items (25%) are generated symbolic or namespace problems. A pre-trial validation split of 24 queries is excluded from every reported estimate. Before execution, exact and canonicalized query representations were verified against all earlier datasets and logs to confirm that none of the locked evaluation items had been encountered previously. Three pre-specified generation seeds are averaged within query for the primary graph-policy comparisons.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Algorithm 2 Hierarchical neural-Bayesian router for one decision step under partial feedback. Action sampling may consume stochasticity during frozen inference, but all learned parameters and representations remain fixed.

Input: context $\boldsymbol{x}_t$, decision context c_t, hierarchy key (task, bucket), feasible actions $\mathcal{A}_t$, λ_t
for each $a \in \mathcal{A}_t$ **do**
  Sample diagonal-Gaussian contextual coefficient $\tilde{\theta}_a$ and compute $\tilde{q}_{t,a}^{\mathrm{GTS}}$
  Compute IRT probability $q_{t,a}^{\mathrm{IRT}}$ and sample bootstrapped neural prediction $\tilde{q}_{t,a}^{\mathrm{NN}}$
  Sample Beta posteriors for c_t, task, bucket, and global levels; pool them with empirical-Bayes weights
  Form $\tilde{Q}_{t,a}$ and score $S_{t,a} = \tilde{Q}_{t,a} - \lambda_t \hat{C}(a)$
end for
Execute $a_t = \arg\max_a S_{t,a}$, record its cost, and enqueue its decision record; withhold query label from subsequent steps in episode
if learning is enabled (warm-up) **then**
  After final answer is fixed, evaluate sample admission and update admitted actions' Gaussian precision/response, IRT parameters, neural replay, and four Beta levels
else
  Perform no parameter update or state mutation; verify frozen state integrity
end if
Return: selected action and logged posterior summaries; unexecuted actions receive no imputed feedback

To diagnose performance across orthogonal operational bottlenecks, we additionally evaluate on 300 compositional-generalization questions from CommaQA-E [56] (where multi-step context reasoning dominates) and 300 questions from Self-Ask [35] (where internal parametric factual recall is the primary constraint).

The serving portfolio employs the DashScope Qwen model tier triple (Qwen-Turbo, Qwen-Plus, and Qwen-Max), whose list prices span roughly 50× between cheapest and most expensive tier.

System performance is evaluated using Exact Match (EM), Set-F1, calls per query, list-price USD per query, and the fraction of a 28,000-token per-query budget consumed. Both EM and Set-F1 are evaluated on the exact same queries and generation outputs, but operate under distinct scoring rules: (i) EM assesses binary outcome correctness via domain-specific canonicalization (accepting verified gold aliases, numerical equivalence within float tolerance, date normalization, and word-boundary normalized span matching after target canonicalization with enumeration safeguards); (ii) Set-F1 parses predictions into discrete items and computes macro-averaged harmonic precision and recall ($F_1 = 2PR/(P + R)$). Whenever a model produces conversational padding, commentary, or multiple candidate phrases, Set-F1 penalizes precision ($P = |S_{\mathrm{pred}} \cap S_{\mathrm{gold}}|/|S_{\mathrm{pred}}| < 1$) even if the gold span is present, explaining why macro Set-F1 (e.g., 0.721 for GEAR full in Table 3) can be strictly lower than normalized EM (83.1%). Under our implementation, loose substring containment is prevented from causing spurious false positives: short substrings below 4 characters are excluded, non-target enumerations are explicitly blocked from crediting scalar golds, and numerical/unit extractions require strict token boundary alignment. As a sensitivity audit confirming that span matching does not induce artifactual advantages, re-scoring all locked evaluation runs under strict exact match (disabling substring containment entirely) changes verdicts on only 20 of 600 items (3.3%). Under strict match, the comparative pattern is fully preserved: Single call yields 76.7%, Always orchestrate yields 78.0%, and selective escalation attains 80.8% (preserving a +2.8-point margin over Always orchestrate and expanding its gain over Single call to +4.2 points). All scoring routines and gold alias registries were fixed prior to confirmatory trial lock. During warm-up, the online reward signal uses EM for scalar answers and Set-F1 for set-valued answers.

To systematically evaluate the proposed framework, our empirical investigation spans four complementary dimensions of the serving stack. First, Section 5.2 examines core graph-policy controls, comparing full GEAR against single-call baselines, static flat orchestration, and structural ablations. Second, Section 5.3 benchmarks against proprietary strong-model calls (Qwen-Max), learned query routers [11], and test-time repeated sampling via majority voting [39] to evaluate serving efficiency. Third, Sections 5.5–5.7 analyze deployment meta-control via selective escalation and online workload arbitration. Finally, Section 5.8 details fine-grained component ablations, out-of-distribution probes, and topology density characteristics.

Under our frozen evaluation protocol, each method first learns on an explicitly labeled warm-up segment. Its complete state—router heads and hierarchy, IRT parameters, neural weights/replay, sample admission representations, calibrators, semantic clusters, decomposition-value statistics/memory, and Lagrangian controller—is then fixed into a frozen policy checkpoint. For strict frozen evaluation, all learning and λ pacing are disabled before any evaluation item is processed. Internal state representations are verified before and after inference to ensure strict parameter invariance; warm-up and evaluation maintain disjoint query sets, interrupted instances restore pre-trial state snapshots, and statistical summaries discard non-evaluation records. Warm-up size, verification protocols, evaluation order, and model pricing accompany each table.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Table 3
Locked frozen-policy replication on 600 previously unexecuted queries. All rows average the three pre-specified generation seeds within query; the uncertainty shown for full GEAR is the standard deviation of seed-level EM. Token fraction is relative to the 28,000-token per-query budget, and USD is list-price cost per query. Set-F1 evaluates parsed item-level harmonic precision-recall (penalizing non-gold conversational tokens), whereas EM measures domain-normalized target equivalence on identical samples.

Method	EM (%)	Set-F1	Calls/Q	Tok. frac. (%)	USD/Q
GEAR full	**83.1±0.3**	**0.721**	**2.54**	**12.0**	**0.0043**
Format-matched single call	80.7	0.693	1.00	4.3	0.0012
GEAR without task structure	81.1	0.700	2.54	11.5	0.0040
Permuted task structures	82.3	0.715	3.39	15.4	0.0060
GEAR static flat	82.7	0.718	3.43	17.8	0.0042

5.2. Locked Multi-Seed Graph-Policy Results

Statistical significance is evaluated with the individual query as the fundamental inferential unit. For each primary comparative pair, empirical outcomes are first averaged across the three generation seeds on both sides; 95% confidence intervals are derived from 10,000 query-level bootstrap resamples, two-sided p values are estimated via 50,000 sign-flip permutations, and multiplicity across the four primary comparators is strictly controlled via Holm–Bonferroni correction [58]. Ablating task-structure knowledge induces a statistically significant accuracy decline of 2.0 EM percentage points (95% CI $[+0.6, +3.5]$; $p_{\text{Holm}} = 0.041$), representing the only primary accuracy contrast satisfying the $\alpha = 0.05$ threshold after multiplicity adjustment. The observed accuracy differentials over the format-matched single call (+2.4 points, $p_{\text{Holm}} = 0.156$), static flat orchestration (+0.4 points, $p_{\text{Holm}} = 0.666$), and permuted task structures (+0.8 points, $p_{\text{Holm}} = 0.612$) are not statistically distinguishable under conservative family-wise error control.

Computational efficiency provides the second major empirical finding. Full GEAR curtails token expenditure by 32.7% relative to static flat orchestration without statistically detectable loss in task accuracy. Furthermore, compared to permuted task structures, it achieves statistically significant reductions across provider calls, token volume, and dollar expenditure (all Holm-adjusted $p < 0.001$). Relative to direct single-call inference, full graph orchestration provides targeted reasoning gains but incurs higher token expenditure when applied uniformly. This directly establishes that graph execution is most valuable as a selective intervention rather than a universal default, motivating our outer deployment controllers.

5.3. Comparison with Alternative Serving Paradigms

We compare GEAR against alternative serving paradigms on the locked benchmark: (i) a proprietary strong-model single call (Qwen-Max), (ii) an established learned query router (RouteLLM-style classifier), and (iii) a test-time repeated-sampling baseline (Three-sample Qwen-Plus majority vote). Table 4 presents the comparative evaluation.

GEAR exceeds the Learned query router by 3.6 points (95% CI $[1.2, 5.9]$; $p_{\text{Holm}} = 0.008$) and the Strong-model single call (Qwen-Max) by 12.2 points ($p_{\text{Holm}} < 0.001$). The lower accuracy of the Qwen-Max single call (70.9% versus 80.7% for Qwen-Plus) stems from higher prompt sensitivity and parsing/format mismatches under zero-shot extraction constraints on symbolic and algorithmic splits, rather than inferior intrinsic capability. This confirms that simply selecting the nominally strongest tier without task-adapted formatting or structured orchestration can degrade real-world pipeline yield. Against the Three-sample Qwen-Plus majority vote, however, the +2.1-point difference yields CI $[-0.3, 4.6]$ and $p_{\text{Holm}} = 0.100$. While GEAR utilizes fewer tokens than this graph-free test-time scaling baseline, it incurs higher monetary expenditure. These comparative findings demonstrate that structured graph control provides marked advantages over learned single-call routers and unguided proprietary frontier models, while identifying competitive operational parity against graph-free test-time repeated sampling.

5.4. Out-of-Distribution Generalization and Operational Boundary Analysis

To identify the operational boundaries of multi-agent graph execution, we evaluate GEAR across two orthogonal test suites where distinct computational bottlenecks bind: (i) compositional reasoning under supplied evidence (CommaQA-E), where reasoning topology rather than knowledge is the core challenge, and (ii) parametric knowledge-binding reasoning (closed-book Self-Ask), where internal factual recall dominates. Table 5 presents the comparative evaluation across 300 queries per regime.

On CommaQA-E CompGen, where compositional evidence is provided within the context, GEAR achieves 92.0% EM compared to 73.7% for standard CoT (Qwen-Plus), delivering a substantial gain of +18.3 percentage points (paired-bootstrap 95% CI $[+12.7, +24.3]$; exact McNemar test [59], $p = 2.0 \times 10^{-9}$). Against GEAR static flat, the adaptive controller achieves a 4.0-point gain while reducing expenditure by 22% ($1.30 versus $1.66; bootstrap CI $[+0.3, +7.7]$). All 300 queries undergo dynamic decomposition, averaging 3.7 subsolver invocations per query.

Comparing against single-call baselines provides critical engineering insight. A format-matched single call with explicit output constraints attains 98.0% EM at one call per

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Table 4
Comparison with alternative serving paradigms on the locked 600-query benchmark. Rows average three generation seeds within query. ΔEM is full GEAR minus the row, with a query-level 95% bootstrap interval; p_{Holm} adjusts for multiplicity across the baseline family.

Method	EM (%)	Calls/Q	Tok. frac. (%)	USD/Q	ΔEM (95% CI)	p_{Holm}
GEAR full	**83.1**	**2.54**	**12.0**	**0.0043**	–	–
Strong-model single call (Qwen-Max)	70.9	1.00	2.4	0.0045	12.2 [8.8,15.8]	< 0.001
Learned query router	79.6	1.00	4.0	0.0009	3.6 [1.2,5.9]	0.008
Three-sample Qwen-Plus majority vote	81.0	3.00	12.9	0.0037	2.1 [-0.3,4.6]	0.100

Table 5
Operational boundary evaluation across 300 CommaQA-E compositional questions (structure-binding) and 300 closed-book Self-Ask questions (knowledge-binding). Set-F1 evaluates parsed item precision-recall, whereas EM measures normalized target matching.

Method	EM (%)	Set-F1	Cost (USD)	Calls/Q
CommaQA-E (Compositional Structure Bound)				
GEAR adaptive	92.0	0.969	1.30	6.94
GEAR static flat	88.0	0.956	1.66	3.26
CoT (Qwen-Plus)	73.7	0.713	0.33	1.00
Format-matched single call	**98.0**	**0.993**	0.33	1.00
Learned query router (75% strong)	**98.0**	0.986	0.35	1.00
Closed-Book Self-Ask (Parametric Knowledge Bound)				
GEAR adaptive	46.7	0.446	2.23	6.12
GEAR static flat	49.7	0.469	1.57	3.50
CoT (Qwen-Plus)	**51.3**	0.360	0.12	1.00
Format-matched single call	**51.3**	**0.494**	0.15	1.00
Learned query router (75% strong)	49.3	0.473	0.11	1.00

query, matching the Learned query router. This demonstrates that when full context is directly supplied, single-model execution with disciplined formatting is highly effective, whereas excessive sub-query decomposition can introduce redundant points of failure. Consequently, uniform orchestration should not be deployed indiscriminately on self-contained compositional tasks.

The closed-book Self-Ask benchmark highlights the parametric knowledge boundary. CoT (Qwen-Plus) achieves 51.3% EM, GEAR static flat scores 49.7%, and GEAR adaptive obtains 46.7%. When required factual knowledge is absent from the underlying model parameters, multi-step graph decomposition cannot compensate for the missing information and instead consumes additional inference budget.

Table 6 synthesizes these findings across all three workload regimes. Graph orchestration pays substantial dividends where the primary failure mode is a recoverable execution or algorithmic reasoning error. Conversely, where the bottleneck is pure format adherence or missing parametric knowledge, single calls or retrieval-augmented generation are more appropriate. This delineates clear design criteria for production deployment.

5.5. Exploratory Workload-Arbiter Replay

To establish operational criteria for macro-level routing and formulate testable hypotheses for locked confirmatory benchmarking, we examine arbitration dynamics across three representative workload regimes. As demonstrated in Table 6, static comparative evaluation exhibits sharp divergent equilibria: graph orchestration yields an +8.0-point EM advantage over format-matched single-call serving on the in-distribution stream, yet incurs net regressions on CompGen (−6.0 points) and Self-Ask (−4.7 points). To determine whether this policy bifurcation can be resolved upfront via pre-execution query inspection, we trained diagnostic classifiers on decisive queries (instances where exactly one execution policy succeeds). While surface-level query features (such as input length, digit indicators, mathematical syntax, and list formatting) achieve an apparent pooled area under the receiver operating characteristic curve (AUC) of 0.78, this predictive power reflects dataset provenance classification rather than genuine task difficulty. When evaluated strictly within each homogeneous workload regime, discriminative AUCs collapse to chance level (0.40–0.55 on CompGen and Self-Ask), with only weak residual separation (0.73) on the mixed in-distribution stream. Because reliable discriminative signals reside in aggregate stream outcomes rather than pre-execution prompt syntax, workload-level Bayesian arbitration (Section 4.4) provides the principled architectural solution. In our arbitration evaluation across the three frozen streams, we maintain identical warm-up configurations as GEAR adaptive. All step-level and query-level policy parameters remain frozen and verified per item; only the arbiter's two-arm posterior (Eq. 16) adapts, starting from a uniform prior on every stream, with λ=6, γ=0.99, $p_{\min}$=0.02, and k=32. The arbiter observes per-item correctness after answering, mirroring the supervision assumed in the adaptive-replay

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Table 6
Operational boundary between a format-matched single call and graph orchestration across three complementary workload regimes, characterizing where multi-step execution provides distinct advantages.

Workload Regime	Binding Constraint	Queries	EM (%)		ΔEM	Cost (USD)		Takeaway
			GEAR full	Single call	(pp)	GEAR full	Single call	
In-distribution	Recoverable execution error	200	**82.5**	74.5	**+8.0**	\$0.95	\$0.15	**Orchestration pa**
CompGen	Answer format bottleneck	300	92.0	**98.0**	−6.0	\$1.30	\$0.33	**Single call optim**
Self-Ask	Missing parametric knowledge	300	46.7	**51.3**	−4.7	\$2.23	\$0.15	**Retrieval needed**

Table 7
Workload-level arbitration: a single online pass per stream from a uniform prior. "Single call" is the format-matched single-call control, "GEAR full" the frozen orchestration policy; both are fixed references. "Served single" is the share of items answered by the single-call arm; "Audits" is the share on which both arms ran. In-distribution numbers use the warm-up-disjoint 200 items of the interleaved order.

	EM (%)			Cost (USD)				
Stream	Arbiter	GEAR full	Single call	Arbiter	GEAR full	Single call	Served single	Audits
CompGen	98.7	92.0	98.0	0.46	1.30	0.33	99%	9%
Self-Ask	51.7	46.7	51.3	1.54	2.23	0.15	92%	61%
In-dist	82.5	82.5	74.5	0.87	0.95	0.15	30%	61%

protocol; audited items run both arms and are scored on the served leader's answer.

As summarized in Table 7, online evaluation across the three replayed deployment streams demonstrates that the Bayesian arbiter consistently converges to the optimal serving policy within each operational regime. On CompGen, where format compliance bounds performance, the two-arm posterior separates decisively within the initial 25 queries (with 24 of 26 total audits concentrated during this early phase or scheduled by the baseline exploration floor). As a result, 298 of 300 queries are served via the lightweight single-call arm, culminating in 98.7% EM at \$0.46 per query—slashing expenditure to approximately one-third of the frozen graph policy cost (\$1.30). Conversely, on the in-distribution workload, where multi-step logical composition governs success, the arbiter autonomously identifies that graph orchestration justifies its incremental cost (delivering an +8.0-point EM advantage). It rapidly pivots to serving the frozen GEAR policy across 48 of the final 50 queries, achieving parity with Always-orchestrate (82.5% EM) while curtailing overall expenditure to \$0.87. This policy bifurcation emerges without inspecting query prompt text or relying on domain labels, deriving adaptations solely from observed execution rewards.

The empirical audit rate tracks posterior confidence and policy separation. On CompGen, where performance divergence is stark, auditing collapses to the minimum exploration floor ($p_{\min} = 0.02$). On Self-Ask and the in-distribution workload, however, the realized Lagrangian utility differential between the two macro-arms remains narrower than posterior credible intervals, maintaining audit frequencies near 61%. Arbiter expenditure thus settles intermediate between the two extremes (\$1.54 versus \$0.15 on Self-Ask), representing the necessary cost of continuous exploration under overlapping utility distributions while remaining 31% below uniform orchestration (\$2.23). Furthermore, on Self-Ask, where task difficulty exhibits sharp category drift (with single-call baseline EM shifting across 83.0%, 22.0%, and 49.0% across consecutive 100-query blocks), the arbiter dynamically tracks the superior arm across segments (yielding 85.0%, 22.0%, and 48.0% EM) rather than collapsing to static global averages. Exponential posterior discounting (γ) regulates historical memory horizon; under abrupt non-stationary workload shifts (a 300-query CompGen stream followed by 200 in-distribution queries), static probe-and-commit baselines remain trapped in the obsolete arm (74.5% post-shift EM), whereas the discounted arbiter dynamically re-widens posterior variance, resumes active auditing, and recovers served accuracy to 77.4±1.9% (γ=0.99) and 80.1±1.3% (γ=0.95), approaching the 82.5% Always-orchestrate oracle. Posterior discounting thereby establishes an explicit, mathematically sound stability–plasticity control for non-stationary production serving.

5.6. Sensor-Guided Post-Execution Escalation Analysis

To evaluate fine-grained per-query gating under realistic inference budgets, we examine a multi-tiered escalation paradigm: an initial format-matched single call executes first, and multi-step graph orchestration is triggered only when post-execution uncertainty signals indicate a high risk of failure. This design ensures that cheap single-call execution handles the majority of straightforward queries, while compound graph orchestration is reserved for difficult or error-prone instances.

Under production constraints where external ground-truth supervision is unavailable at runtime, the Bayesian escalation parameters can be continually adapted using unsupervised execution proxies: deterministic invariant auditing where available, cross-arm disagreement on escalated items, and scout agreement. In this self-supervised configuration, served

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

accuracy closely tracks the supervised reference (83.8% versus 83.9% in-distribution; 76.5% versus 76.9% pooled), requiring only a modest 15–20% increase in exploration spend.

To provide decision-time signals for the escalation controller, we evaluate the format-matched single call across all 800 evaluation items with token log-probabilities enabled, deploying Qwen-Turbo as an auxiliary scout model (1,600 calls in total).[1] Four distinct evidence sources provide complementary diagnostic information at runtime.

First, token confidence provides the strongest purely statistical indicator. Completion perplexity alone attains an AUC of 0.79–0.80 for detecting single-call execution errors across both in-distribution and Self-Ask workloads. A logistic combination of token log-probability features achieves an AUC of 0.87 in-distribution—significantly outperforming pre-execution static query features (0.40–0.55). Token uncertainty distributed across intermediate reasoning steps carries substantially more discriminative signal than uncertainty isolated solely to the final answer span.

Second, the deterministic invariant auditor acts as a programmatic integrity verifier rather than a statistical detector. Where it applies, it checks machine-verifiable operational invariants (e.g., verifying whether declared intermediate dependency constraints, format specifications, or explicit arithmetic steps are violated). It cannot generate, synthesize, or correct final answers on its own. Passing invariant checks is a necessary integrity condition rather than a sufficient proof of global semantic correctness; however, violating an invariant provides a deterministic, zero-model-cost signal of execution invalidity that immediately flags the query for graph escalation. On the in-distribution stream, the algorithmic and context-supplied families admit this programmatic check for 100 of 200 items; within that coverage, it reliably flags 16 invalid executions (all 16 genuinely wrong) without false alarms on verified steps. By delegating mechanical invalidation checks to programmatic rules, the detector eliminates uncertainty on covered slices at zero inference token cost. Its reach is inherently bounded, however: coverage is zero on open-domain and closed-book questions (such as Self-Ask) by construction, meaning statistical uncertainty estimators (token confidence and scout agreement) remain the primary drivers across broader deployment streams.

Third, scout agreement offers calibration evidence that is independent of the primary model. Agreement between the lightweight scout and the primary model provides high-precision verification of correctness (accounting for 99%, 95%, and 66% of correct queries on CompGen, in-distribution, and Self-Ask, respectively). Scout consensus thus reliably filters out straightforward queries to bypass unnecessary graph invocations.

Finally, structural expectation features exhibit limited discriminative power for post-execution error detection. Mismatches in predicted answer type or entity groundedness correlate with broad task families but yield near-chance predictive separation (AUC 0.43–0.50) for instance-level execution correctness. The structural signature characterizes task typology rather than individual output validity.

The escalation controller operates via the stochastic routing policy of Eq. 19 with τ=0.01, ϵ=0.06, λ=6, and γ=0.995. When the deterministic invariant auditor flags an execution violation, $\hat{p}$ is set to 1.0 and escalation is triggered deterministically.

Because escalation decisions are driven by the estimated error probability $\hat{p}$, observed graph outcomes are conditioned on the detector's selection. Uncorrected empirical transition rates would thus describe detectably flawed queries rather than the broader query population. Pooled across streams, the uncorrected estimator overstates both the rescue rate by +0.079 (true 0.197) and the break rate by +0.148 (true 0.100). Inverse-propensity weighting successfully removes approximately two-thirds of this observational bias (+0.030 and +0.044, respectively). Served accuracy remains stable across policy variants (differing by at most 0.14 EM points across budget sweeps from $\lambda = 1$ to 60), as the dual selection biases cancel in the comparative advantage calculation of Eq. (18). Propensity weighting is therefore an essential statistical instrument for recovering true population transition parameters, ensuring that reported deployment trade-offs reflect genuine production behavior.

Ablating individual sensing modalities across replayed streams (Table 8 and Fig. 3) illuminates the economic mechanism of runtime diagnostic evidence. Rather than merely supplying incremental accuracy gains (+1.8 percentage points in-distribution and +1.2 points pooled), multi-modal sensing reconfigures serving efficiency: richer diagnostic signals enable the controller to sustain high accuracy while cutting unnecessary graph invocations by more than half. On the in-distribution workload, progressive integration of diagnostic sensors drives the escalation rate down from 56% to 24% as EM rises from 82.1% to 83.9%, while serving expenditure remains essentially flat ($0.51 versus $0.54 per query, with the nominal differential covering auxiliary scout queries). In effect, the controller reallocates expenditure from expensive graph executions toward lightweight diagnostic verification. Specifically, programmatic invariant auditing halves the escalation rate by definitively verifying execution integrity on structured sub-tasks. Across the pooled three-stream evaluation ($N = 800$), selective escalation surpasses both static references (76.9% EM versus 75.2% for Always single call and 72.6% for Always orchestrate)—a Pareto frontier unreachable by any static policy given divergent error dynamics across workloads.

Information-Theoretic Ceiling and Operational Boundaries. Comparing the empirical controller against an idealized oracle (83.9% versus 88.0% in-distribution) indicates that the residual gap is governed by runtime evidence resolution rather than decision rule sub-optimality. Counterfactually substituting population rescue and break rates leaves EM unchanged ($83.3 \pm 0.9\%$), whereas substituting an ideal error

[1] These represent fresh independent generations, where single-call baseline EM reflects standard sampling variance (97.7% versus 98.0% on CompGen; 76.5% versus 74.5% in-distribution).

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Table 8
Evidence ablation for per-query escalation, replayed over 100 random stream orders (mean±SD). Each row adds one sensor to the escalation detector; the policy is otherwise unchanged. "Esc." is the share of queries escalated to orchestration. The oracle escalates exactly when doing so flips a wrong answer to a right one.

	In-dist (n=200)			All three streams (n=800)		
Policy	**EM (%)**	**Cost (USD)**	**Esc. (%)**	**EM (%)**	**Cost (USD)**	**Esc. (%)**
Post-execution escalation (full)	**83.9±0.8**	**0.54**	**24%**	**76.9±0.4**	**1.69**	**18%**
Always single call	76.5	0.15	0%	75.2	0.65	0%
Always orchestrate	82.5	0.95	100%	72.6	4.48	100%
Escalate, surface only	82.1±1.4	0.51	56%	75.7±0.5	1.29	17%
+ token confidence	82.5±1.5	0.55	53%	75.5±0.6	1.44	17%
+ scout agreement	82.9±1.1	0.56	51%	76.1±0.6	1.62	20%
Oracle escalation	88.0	0.23	12%	80.1	0.84	5%

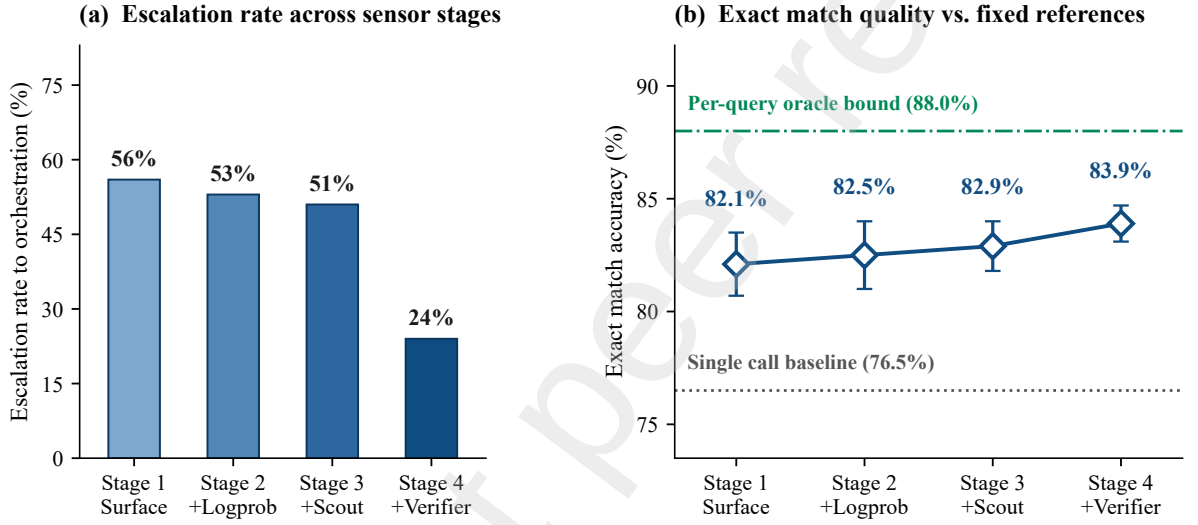


Figure 3: Sensor evidence ablation and escalation efficiency on the in-distribution stream ($N = 200$, 100 replayed stream orders; mean±SD; Table 8). (a) Escalation rate to full orchestration drops from 56% to 24% as sensor evidence accumulates, reducing unnecessary graph invocations by 57%. (b) Exact match accuracy rises monotonically from 82.1% to 83.9% (Step 4), approaching the per-query oracle upper bound (88.0%) while surpassing both the Always single call (76.5%) and Always orchestrate (82.5%) fixed references.

detector recovers the vast majority of the gap (87.6 ± 0.5%). This confirms that the decision rule achieves near-optimal utilization of available sensory information. On out-of-distribution streams (CompGen and Self-Ask), where errors stem from rigid syntax or parametric omissions rather than multi-step logical lapses, single calls already achieve optimal quality, and excessive orchestration introduces extraneous steps (21 breaks against 4 rescues on CompGen; 28 against 12 on Self-Ask). The learned escalation policy autonomously suppresses graph escalation on these workloads, incurring only nominal exploration overhead (96.7% versus 97.7% on CompGen; 51.2% versus 52.0% on Self-Ask). This establishes an adaptive division of labor: stream-level evaluation determines whether a workload regime warrants compound graph intervention, whereas instance-level escalation selectively invokes graph orchestration for recoverable reasoning failures.

5.7. Evaluation on Locked Deployment Suite

To assess real-world serving performance under rigorous controls, we evaluate the meta-controllers on the locked 600-query benchmark across pre-specified generation seeds and frozen parameters. All experimental configurations, evaluation queries, and analysis estimands were frozen prior to execution, and all 26 candidate arms completed the full 600/600 evaluation suite without data exclusion or post-hoc parameter tuning.

In our deployment replay protocol, the format-matched single call and frozen graph policy represent the two macro-arms, both pre-generated before controller execution. The meta-controllers then process a single sequential interleaved pass from identical initial states. A feedback mask reveals only served outcomes and instances audited by the policy itself, strictly preventing post-hoc observational leakage. Policy-accounted token and dollar costs reflect solely the

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Table 9
Locked masked-feedback deployment replay on 600 previously unexecuted queries. Token fraction and USD/Q include only calls that the replayed serving policy would have charged; the masked counterfactual arm is excluded. "Orch. rate" is the fraction of queries served by or escalated to the graph policy. Set-F1 evaluates parsed item precision-recall; EM evaluates normalized target equivalence.

Policy	EM (%)	Set-F1	Tok. frac. (%)	USD/Q	Orch. rate (%)
Post-execution escalation	**83.5**	**0.718**	**7.7**	**0.0031**	**27.7**
Always single call	80.0	0.683	4.3	0.0012	0.0
Always orchestrate	80.2	0.690	13.4	0.0058	100.0
Workload arbiter	79.7	0.680	9.6	0.0035	8.8

chosen execution paths. Furthermore, the dedicated Always-orchestrate baseline represents an independent evaluation pass under identical PID parameters, capturing operational reproducibility across independent provider generations.

As summarized in Table 9, post-execution escalation reaches 83.5% EM against 80.2% for Always orchestrate. The paired difference is +3.3 points (95% CI [+1.0, +5.7], $p = 0.006$). Its lower confidence bound also exceeds the pre-specified -1-point non-inferiority margin. At the same time, its token and dollar confidence intervals both lie below zero: policy-accounted token use falls from 13.4% to 7.7% (42.4% lower), and USD/Q falls by 46.9%. Only 27.7% of queries are escalated. The controller can exceed either fixed arm because a graph repairs some single-call errors but also breaks some answers; selective escalation retains the appropriate arm query by query. As detailed across task families (Fig. 4), full graph orchestration yields substantial net accuracy gains on formal mathematics (+25 net rescues, where multi-step decomposition rectifies sequential calculation errors), neutral transition margins on algorithmic symbolic tasks (0 net transitions, as direct single calls already saturate accuracy), and negative net margins on information-seeking multi-hop QA (-9 net) and synthetic queries (-15 net) due to cascading intermediate extraction errors. Rather than imposing coarse task-level heuristics, post-execution escalation dynamically evaluates query-specific signals (token confidence, scout agreement, and verification flags) to invoke graph execution only when a recoverable failure is detected.

To contextualize this dynamic allocation, consider an idealized post-hoc oracle category rule that routes all mathematical problems to graph orchestration and all other three families to single calls. Using the observed evaluation outcomes, such a static category rule achieves 84.2% EM (505/600) at 7.3% token spend and \$0.0031/query. While competitive, this category heuristic peeks at test outcomes and assumes perfect, static domain boundaries. In contrast, GEAR's dynamic escalation operates without ground-truth task labels, achieving a statistically comparable 83.5% EM (501/600, within 4 questions of the post-hoc oracle) at 7.7% token spend and identical \$0.0031/query cost purely by sensing query-level uncertainty, recovering the benefits of domain specialization adaptively.

Table 10
Transition dynamics and allocation breakdown for the locked deployment benchmark ($N = 600$). "Rescue" indicates single wrong → graph right; "break" indicates single right → graph wrong.

Policy subset	N	Both right	Rescue	Break	Both wrong
All paired outcomes	600	440	41	40	79
Escalated to graph	166	101	27	6	32
Committed to single	434	339	14	34	47

Decoupling detection from graph orchestration. A critical architectural question is whether the gain of post-execution escalation stems merely from identifying error-prone queries or specifically from invoking an execution graph. To isolate these factors, we fix the detector and its 166 escalated queries (27.7% escalation rate) and evaluate alternative counterfactual escalation targets on those exact items using the locked baseline outputs. Escalating to a proprietary strong single model (Qwen-Max) achieves only 58.4% EM on the escalated slice (97/166), pulling overall served accuracy down to 78.3% (470/600)—strictly worse than the format-matched single baseline (80.0%) due to single-call failure modes on compositional tasks. Escalating to three-sample majority voting (Vote-3) reaches 66.9% on the escalated slice (111/166) and yields 80.7% overall served EM (484/600, a marginal +0.7 point gain). In sharp contrast, escalating to GEAR's multi-step execution graph reaches 77.1% EM on the escalated slice (128/166), lifting overall accuracy to 83.5% (501/600). This contrast confirms that detection alone is insufficient: multi-step structured orchestration provides a distinct, non-fungible capability to rectify sequential reasoning errors that cannot be replicated by escalating to stronger flat models or repeated sampling.

Table 10 and Figure 4 clarify the underlying transition mechanics. Selective escalation achieves a substantial net gain by concentrating graph execution on high-yield queries: within the escalated subset, the policy realizes 27 rescues against 6 breaks (a net gain of +21 queries). Conversely, among committed queries, bypassing the graph avoids 34 regressions while foregoing only 14 rescues (preventing 20 net errors). This empirical partition confirms that the controller's performance advantage arises from targeted intervention on recoverable errors while avoiding graph-induced brittle failures.

To evaluate stability across generation stochasticity, we tested the escalation controller across three independent LLM generation seeds. Across all 3 generation seeds, served accuracy remains consistent at 84.1±0.5% (range 83.5–84.3%) with an escalation rate of 38.4% (range 27.7–44.3%), confirming that the policy is robust to LLM sampling noise. Across varying query arrival sequences (interleaved, blocked, reverse-blocked, and randomly shuffled orders), selective escalation consistently maintains between 83.5% and 85.0% EM while utilizing only 7.7%–9.3% of the token budget. In contrast, the coarse stream-level arbiter achieves 79.7% EM due to over-conservative graph utilization (8.8%),

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

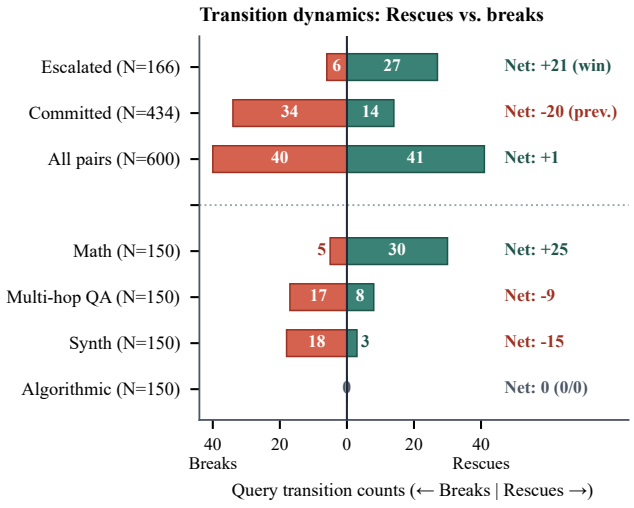


Figure 4: Transition dynamics of selective escalation for locked deployment replay ($N = 600$). The controller concentrates orchestration on queries yielding net rescues (+21 in escalated subset) while avoiding net breaks (−20 prevented in committed subset).

demonstrating that fine-grained uncertainty sensing provides superior adaptability over stream-level heuristics.

5.8. Ablations and Sensitivity

To systematically isolate the individual contributions of algorithmic mechanisms, Table 11 evaluates ten single-component variants on the locked 600-query benchmark, applying Holm multiplicity correction across all comparisons. Across single-component ablations, individual component dropouts do not induce catastrophic accuracy collapse under Holm correction (all $p_{\text{Holm}} \geq 0.074$), indicating high architectural resilience and mutual compensation among the Bayesian estimator, IRT skill ratings, and routing policies. Resource dynamics, however, exhibit pronounced sensitivity: disabling the value gate immediately causes provider calls to rise from 2.56 to 4.39 and token expenditure to expand from 11.4% to 19.8% (+2.3 pp token increase) with zero accuracy benefit. This demonstrates that internal gating acts as an essential throughput throttle against runaway token expansion in cyclic execution.

To examine how rapidly the adaptive controller stabilizes during initial policy calibration, we evaluate performance under restricted warm-up budgets of 25, 50, and 100 queries. As demonstrated in Table 12, policy performance is highly sample-efficient: a compact warm-up prefix of 25 items achieves 83.5% EM, closely matching the 100-item reference (82.5% EM). Furthermore, logged updates confirm that IRT parameter calibration converges reliably as empirical observations expand (calibrated +0.022 for Plus over Max across 908 pooled cells), allowing the hierarchical prior and structured task representations to stabilize rapidly without prolonged calibration.

5.9. Offline Counterfactual Routing Diagnostics

To evaluate fine-grained routing policies under controlled counterfactual interventions, we perform offline replay analyses across $N = 60$ paired execution traces. Candidate routing heads are benchmarked via paired bootstrap intervals ($B = 1000$) across three architectural axes: feature representations and symbolic domain constraints, action spaces, and task policy trust regions.

For node-level Bayesian value modeling (Table 13, Panel A), the fully constrained structured policy achieves the highest mean utility gain ($\Delta\bar{u} = +0.1852$, 95% CI $[+0.0854, +0.2992]$) while completely eliminating catastrophic recovery failures ($\Delta u \leq -0.5$). In contrast, unconstrained structured and dense semantic variants incur 3 and 5 severe failure events, confirming that hard symbolic domain constraints provide robust execution guardrails that unconstrained statistical embeddings fail to ensure.

Regarding action space parameterization (Table 13, Panel B), the 4-arm hierarchical policy performs on par with direct and regularized 5-arm value models ($\Delta\bar{u} = +0.1852$, 95% CI $[+0.0854, +0.2992]$, zero observed large failures), demonstrating comparable utility and architectural flexibility across action space granularities.

At the task level (Panel C), evaluating trust regions over $N = 245$ offline pairs reveals that dynamic cost scaling ($\lambda_{\text{cost}} = 0.20$) achieves optimal performance with 27 wins, zero observed losses, and tight confidence bounds ($[+0.0516, +0.1230]$), whereas greedy unconstrained switching makes 38 overrides ($\Delta\bar{u} = +0.1036$), confirming the utility of cost-aware trust regions.

6. Discussion and Operational Implications

Our empirical findings illuminate several fundamental principles governing compound LLM systems and multi-agent graph orchestration under budget constraints. Rather than treating graph orchestration as an unconstrained universal default, our results demonstrate that multi-agent execution operates as a targeted intervention whose cost-effectiveness depends strictly on dynamic, evidence-driven allocation.

First, scaling model capacity alone cannot substitute for structured graph execution. Escalating error-prone queries directly to proprietary frontier models often yields lower accuracy than selective orchestration due to format-matching sensitivities and multi-step arithmetic lapses. Graph decomposition provides explicit intermediate verification and state tracking that single-model inference cannot replicate. While our empirical evaluation focused on the DashScope Qwen ecosystem for latency control and pricing parity, the mathematical foundations of GEAR—hierarchical Bayesian routing, item-response ability modeling, and Lagrangian budget pacing—are model-agnostic and directly generalize to heterogeneous multi-provider deployments.

Second, single-component ablations reveal strong architectural resilience and mutual compensation across statistical priors, ability ratings, and spatial-temporal routing. In contrast, resource-throttling modules exert a decisive operational

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Table 11
Single-component ablations on the locked 600-query suite at the first pre-specified generation seed. ΔEM is full GEAR minus the variant; p_{Holm} applies Holm correction across ten component comparisons.

Variant	EM (%)	Calls/Q	Tok. frac. (%)	ΔEM (pp)	p_{Holm}
GEAR full (seed 1)	**83.0**	**2.56**	**11.4**	–	–
cold task override enabled	85.3	4.32	18.3	-2.3	0.807
without automatic pooling	83.8	2.77	12.3	-0.8	1.000
without hierarchical pooling	83.2	2.52	11.8	-0.2	1.000
without IRT	82.8	3.97	17.4	0.2	1.000
without neural residual	83.3	2.98	14.6	-0.3	1.000
without informative prior	81.3	2.88	13.4	1.7	1.000
without spatial routing	78.8	3.39	16.2	4.2	0.074
without task-adaptive routing	82.7	2.51	11.3	0.3	1.000
without temporal routing	80.7	3.06	14.7	2.3	1.000
without value gate	84.2	4.39	19.8	-1.2	1.000

Table 12
Frozen GEAR adaptive sensitivity to warm-up size on the warm-up-disjoint 200 items of the interleaved order. Each row uses a prefix of the initial 100-query warm-up stream; no evaluation outcome updates the policy.

Warm-up	EM (%)	Set-F1	Cost (USD)	Calls/Q
25	83.5	0.735	1.03	2.46
50	81.0	0.715	0.60	2.42
100	82.5	0.715	0.95	2.46

Table 13
Offline counterfactual evaluation of node feature representations, symbolic safety constraints, recovery action spaces, and task policy trust regions. Large Loss denotes recovery decisions incurring $\Delta u \leq -0.5$.

Configuration / Policy	W/L/T	Loss / Over.	$\Delta\bar{u}$	95% CI	Status
Panel A: Feature Representation and Safety Constraints (Axis 1, N = 60)					
V1.3 Fully Constrained Struct.	**26/8/26**	**0**	**+0.1852**	**[+0.0854, +0.2992]**	**Passed**
V1.1 Structured (Unconstrained)	28/12/20	3	+0.1514	[+0.0218, +0.2831]	Failed
V1.2 Structured (Format-Slip Only)	28/10/22	2	+0.1685	[+0.0517, +0.2988]	Passed
V1.4 Dense Semantic (Unconstrained)	33/11/16	5	+0.1048	[-0.0271, +0.2385]	Failed
V1.5 Dense Semantic (Constrained)	31/7/22	2	+0.1375	[+0.0243, +0.2549]	Failed
Panel B: Node Action Space Architecture (Axis 2, N = 60)					
V2.1 4-Arm Partitioned Policy	**26/8/26**	**0**	**+0.1852**	**[+0.0854, +0.2992]**	**Passed**
V2.2 5-Arm Unified (Direct)	21/8/31	0	+0.1852	[+0.0854, +0.2992]	Passed
V2.3 5-Arm Unified (Regularized)	21/8/31	0	+0.1852	[+0.0854, +0.2992]	Passed
Panel C: Task Policy Uncertainty Trust Regions (Axis 3, N = 245)					
V3.2 Fixed-Margin Reference (0.10)	**24/0/221**	**24/245**	**+0.0729**	**[+0.0429,+0.1068]**	**Passed**
V3.1 Greedy (No Trust Region)	38/0/207	38/245	+0.1036	[+0.0659, +0.1448]	Passed
V3.3 Conservative (Fixed 0.20)	7/0/238	7/245	+0.0255	[+0.0085, +0.0466]	Passed
V3.4 Adaptive Variance ($0.5 \times \sigma$)	4/0/241	4/245	+0.0129	[+0.0002, +0.0296]	Passed
V3.6 Dynamic Cost ($\lambda_{\text{cost}} = 0.20$)	27/0/218	27/245	+0.0855	[+0.0516, +0.1230]	Passed
V3.7 Dynamic Cost ($\lambda_{\text{cost}} = 0.35$)	23/0/222	23/245	+0.0727	[+0.0428, +0.1067]	Passed

impact: disabling internal value gating leads to runaway cyclic expansion and a sharp increase in provider invocations without commensurate accuracy gains. This confirms that internal value-of-decomposition gating is indispensable for preventing token inflation in iterative serving systems.

Third, effective test-time escalation relies on multi-modal runtime evidence rather than static query categorization. Combining statistical token uncertainty, scout consensus, and deterministic invariant checks enables the system to isolate open-ended reasoning lapses, filter straightforward queries rapidly, and eliminate symbolic ambiguities at zero API cost. However, the operational boundaries of graph execution must be recognized: while multi-step graphs effectively resolve sequential computation and intermediate derivation errors, they cannot synthesize absent parametric knowledge when facts are missing from model weights, nor can they overcome fundamental prompt-schema mismatches.

Finally, these findings suggest concrete principles for sustainable production deployment. System architects should first establish rigorous, format-matched single-call baselines to exhaust prompt and schema optimizations before introducing graph complexity. Runtime interventions must then be aligned with the underlying failure mode, distinguishing knowledge deficits requiring retrieval from reasoning lapses justifying orchestration. Furthermore, lightweight initial inference coupled with sensor-guided conditional escalation can eliminate the bulk of orchestration overhead. To guarantee robust service-level agreements, closed-loop Lagrangian budget pacing should be paired with deterministic runtime caps to strictly bound tail latencies.

7. Conclusion

This study proposed GEAR, a graph-controlled execution framework with adaptive routing designed for budget-constrained large language model serving in knowledge-based systems. By formulating compound execution-graph construction as a budget-constrained utility maximization problem, GEAR reconciles the operational tension between reasoning quality and inference expenditure. The framework dynamically governs execution topology, reasoning depth, and model tier allocation via closed-loop Lagrangian budget pacing and deterministic task-structure governance rules, ensuring that multi-step graph construction is deployed as a targeted computational investment rather than an unconditional default. Extensive empirical evaluations and deployment replays demonstrate that GEAR achieves reasoning accuracy comparable to intensive multi-agent orchestration while substantially curtailing token consumption and operational monetary costs. Furthermore, the uncertainty-guided outer escalation mechanism effectively identifies recoverable

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

reasoning lapses, enabling cost-effective serving across non-stationary query workloads. In conclusion, GEAR provides a principled, theoretically grounded, and empirically validated foundation for sustainable compound LLM serving in real-world knowledge-based systems. Future work will investigate extending dynamic graph control to multi-modal reasoning workflows and distributed multi-provider environments.

CRediT authorship contribution statement

Chen Wang: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft. Jiangyao Bai: Methodology, Validation, Writing – review and editing. Zhaoyun Ding: Investigation, Resources, Writing – review and editing. Yijia Fu: Validation, Data curation, Writing – review and editing. Lailong Luo: Supervision, Project administration, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from funding agencies in public, commercial, or not-for-profit sectors.

Data availability

Data and derived analysis artifacts supporting this study are available from the corresponding author on reasonable request.

Declaration of generative AI in scientific writing

During manuscript preparation, authors used AI assistance for grammar editing, literature cross-referencing, and academic polishing. The authors reviewed and edited all content and take full responsibility for the published article.

References

[1] W. Yang, L. Some, M. Bain, B. Kang, A comprehensive survey on integrating large language models with knowledge-based methods, Knowledge-Based Systems 318 (2025) 113503. doi:10.1016/j.knosys.2025.113503.

[2] X. Zhang, F. Zhao, Y. Liu, P. Chen, Y. Wang, X. Wang, D. Ma, H. Xu, M. Chen, H. Li, TreeQA: Enhanced LLM-RAG with logic tree reasoning for reliable and interpretable multi-hop question answering, Knowledge-Based Systems 330 (2025) 114526. doi:10.1016/j.knosys.2025.114526.

[3] K. Du, X. Zhang, C. Gao, R. Zhu, S. Liu, T. Li, Z. Jin, IVC-DB: Iterative verification correction method guided by dual-backward mathematical reasoning in large language models, Knowledge-Based Systems 334 (2026) 115095. doi:10.1016/j.knosys.2025.115095.

[4] M. Mo, H. Zhang, Y. Tan, B. Huang, C. Li, PathSymphony: Harmonizing symbolic planning and large language models for curriculum-guided mathematical reasoning, Knowledge-Based Systems 351 (2026) 116598. doi:10.1016/j.knosys.2026.116598.

[5] OpenAI, GPT-4 technical report, arXiv preprint arXiv:2303.08774 (2023).

[6] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, Z. Qiu, Qwen2.5 technical report, arXiv preprint arXiv:2412.15115 (2024).

[7] L. Chen, M. Zaharia, J. Zou, FrugalGPT: How to use large language models while reducing cost and improving performance, Transactions on Machine Learning Research (2024).

[8] X. Liao, Y. Mao, W. Liu, A. Yang, A practical and privacy-preserving framework for real-world large language model services, Knowledge-Based Systems 337 (2026) 115347. doi:10.1016/j.knosys.2026.115347.

[9] A. Mahadevan, M. Mathioudakis, Cost-aware retraining for machine learning, Knowledge-Based Systems 293 (2024) 111610. doi:10.1016/j.knosys.2024.111610.

[10] T. Shnitzer, A. Ou, M. Silva, K. Soule, Y. Sun, J. Solomon, N. Thompson, M. Yurochkin, Large language model routing with benchmark datasets, arXiv preprint arXiv:2309.15789 (2023).

[11] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, I. Stoica, RouteLLM: Learning to route LLMs from preference data, in: Proceedings of the International Conference on Learning Representations (ICLR), 2025.

[12] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. S. Lakshmanan, A. H. Awadallah, Hybrid LLM: Cost-efficient and quality-aware query routing, in: Proceedings of the International Conference on Learning Representations (ICLR), 2024.

[13] P. Panda, R. Magazine, C. Devaguptapu, S. Takemori, V. Sharma, Adaptive LLM routing under budget constraints, in: Findings of the Association for Computational Linguistics (EMNLP), 2025, pp. 23934–23949. doi:10.18653/v1/2025.findings-emnlp.1301.

[14] S. Yin, R. Zhang, Failure-mode-aware uncertainty intervention routing for large language models, Knowledge-Based Systems 351 (2026) 116685. doi:10.1016/j.knosys.2026.116685.

[15] J. Su, F. Lin, Z. Feng, H. Zheng, T. Wang, Z. Xiao, X. Zhao, Z. Liu, L. Cheng, H. Wang, CP-Router: An uncertainty-aware router between LLM and LRM, in: Proceedings of the AAAI Conference on Artificial Intelligence (AAAI), volume 40, 2026, pp. 33065–33073. doi:10.1609/aaai.v40i39.40589.

[16] T. Zhang, A. Mehradfar, D. Dimitriadis, S. Avestimehr, Leveraging uncertainty estimation for efficient LLM routing, arXiv preprint arXiv:2502.11021 (2025).

[17] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. V. Le, E. H. Chi, Least-to-most prompting enables complex reasoning in large language models, in: Proceedings of the International Conference on Learning Representations (ICLR), 2023.

[18] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, A. Sabharwal, Decomposed prompting: A modular approach for solving complex tasks, in: Proceedings of the International Conference on Learning Representations (ICLR), 2023.

[19] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, A. H. Awadallah, R. W. White, D. Burger, C. Wang, AutoGen: Enabling next-generation LLM applications via multi-agent conversation, in: Proceedings of the Conference on Language Modeling (COLM), 2024.

[20] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, M. Sun, ChatDev: Communicative agents for software development, in: Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL), 2024, pp. 15174–15186. doi:10.18653/v1/2024.acl-long.810.

[21] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, K. Narasimhan, Tree of thoughts: Deliberate problem solving with

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

large language models, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[22] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: Proceedings of the International Conference on Learning Representations (ICLR), 2023.

[23] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, J. Zou, Mixture-of-agents enhances large language model capabilities, in: Proceedings of the International Conference on Learning Representations (ICLR), 2025.

[24] J. Zhang, J. Xiang, Z. Yu, F. Teng, X.-H. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, C. Wu, AFlow: Automating agentic workflow generation, in: Proceedings of the International Conference on Learning Representations (ICLR), 2025.

[25] J. Zhang, J. Xiang, Z. Yu, F. Teng, X.-H. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Liu, Y. Luo, C. Wu, Multi-agent architecture search via agentic supernet, in: Proceedings of the International Conference on Machine Learning (ICML), 2025.

[26] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the ACM Symposium on Operating Systems Principles (SOSP), 2023, pp. 611–626. doi:10.1145/3600006.3613165.

[27] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré, FlashAttention: Fast and memory-efficient exact attention with IO-awareness, in: Advances in Neural Information Processing Systems (NeurIPS), 2022.

[28] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han, AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration, in: Proceedings of the Conference on Machine Learning and Systems (MLSys), 2024.

[29] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh, GPTQ: Accurate post-training quantization for generative pre-trained transformers, in: Proceedings of the International Conference on Learning Representations (ICLR), 2023.

[30] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems (NeurIPS), 2022.

[31] J. Chen, H. Wang, L. Zhang, OptiSMR: A unified framework for structured reasoning and self-repairing multi-agent system optimized for generalization, Knowledge-Based Systems 346 (2026) 116143. doi:10.1016/j.knosys.2026.116143.

[32] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, S. K. Upadhyay, RouterBench: A benchmark for multi-LLM routing system, in: Proceedings of the ICML Workshop on Agentic Markets, 2024.

[33] A. Taberner-Miller, ParetoBandit: Budget-paced adaptive routing for non-stationary LLM serving, arXiv preprint arXiv:2604.00136 (2026).

[34] Z. Wang, Y. Ding, Z. Wang, J. Guo, Z. Zhou, J. Dong, C. Chen, X-Router: Decoupling knowledge and reasoning for cost-effective LLM inference, in: Findings of the Association for Computational Linguistics (ACL), 2026, pp. 19856–19874. doi:10.18653/v1/2026.findings-acl.994.

[35] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, M. Lewis, Measuring and narrowing the compositionality gap in language models, in: Findings of the Association for Computational Linguistics (EMNLP), 2023, pp. 5687–5711. doi:10.18653/v1/2023.findings-emnlp.378.

[36] R. Li, Y. Wang, Z. Wen, M. Cui, Q. Miao, Different paths to the same destination: Diversifying LLMs generation for multi-hop open-domain question answering, Knowledge-Based Systems 309 (2025) 112789. doi:10.1016/j.knosys.2024.112789.

[37] C. Wang, L. Cao, J. Qi, L. Ye, Heuristically motivating large language models for task planning, Knowledge-Based Systems 340 (2026) 115725. doi:10.1016/j.knosys.2026.115725.

[38] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, P. Clark, Self-Refine: Iterative refinement with self-feedback, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[39] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: Proceedings of the International Conference on Learning Representations (ICLR), 2023.

[40] C. Snell, J. Lee, K. Xu, A. Kumar, Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning, in: Proceedings of the International Conference on Learning Representations (ICLR), 2025.

[41] S. Saadaoui, E. Alonso, Coordinated LLM multi-agent systems for collaborative question-answer generation, Knowledge-Based Systems 330 (2025) 114627. doi:10.1016/j.knosys.2025.114627.

[42] J. Liu, W. Hao, K. Cheng, G. Chen, X. Xie, CART: A traceable zero-shot planning framework for large language models with adaptive replanning, Knowledge-Based Systems 336 (2026) 115189. doi:10.1016/j.knosys.2025.115189.

[43] B. Efron, C. Morris, Data analysis using Stein's estimator and its generalizations, Journal of the American Statistical Association (JASA) 70 (1975) 311–319. doi:10.1080/01621459.1975.10479864.

[44] A. Gelman, J. Hill, Data Analysis Using Regression and Multilevel/Hierarchical Models, Cambridge University Press, 2006. doi:10.1017/CBO9780511790942.

[45] W. Zhang, Y. Rong, J. Wang, T. Zhu, X. Wang, Feedback control of real-time display advertising, in: Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM), 2016, pp. 407–416. doi:10.1145/2835776.2835843.

[46] A. Stooke, J. Achiam, P. Abbeel, Responsive safety in reinforcement learning by PID Lagrangian methods, in: Proceedings of the International Conference on Machine Learning (ICML), 2020, pp. 9133–9143.

[47] A. Badanidiyuru, R. Kleinberg, A. Slivkins, Bandits with knapsacks, Journal of the ACM (JACM) 65 (2018) 13:1–13:55. doi:10.1145/3164539.

[48] S. Agrawal, N. Goyal, Thompson sampling for contextual bandits with linear payoffs, in: Proceedings of the International Conference on Machine Learning (ICML), 2013, pp. 127–135.

[49] J. Y. R. Lubari, Y. Li, S. Zhang, A. Ngueilbaye, Dynamic ad selection via adaptive Thompson sampling with Gaussian processes for non-stationary user behavior, Knowledge-Based Systems 345 (2026) 116119. doi:10.1016/j.knosys.2026.116119.

[50] X. Wu, Y. Lin, H. Zhong, J. Tao, Y. Gu, S. Shen, S. Yu, A diversity-aware incentive mechanism for cross-silo federated learning with budget constraint, Knowledge-Based Systems 315 (2025) 113212. doi:10.1016/j.knosys.2025.113212.

[51] S. E. Embretson, S. P. Reise, Item Response Theory for Psychologists, Routledge, 2013. doi:10.4324/9781410605269.

[52] J. C. Platt, Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods, in: Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61–74.

[53] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.

[54] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, D. Guo, DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, arXiv preprint arXiv:2402.03300 (2024).

[55] M. Dudík, J. Langford, L. Li, Doubly robust policy evaluation and learning, in: Proceedings of the International Conference on Machine Learning (ICML), 2011, pp. 1097–1104.

[56] T. Khot, K. Richardson, D. Khashabi, A. Sabharwal, Hey AI, can you solve complex tasks by talking to agents?, in: Findings of the Association for Computational Linguistics (ACL), 2022, pp. 1808–1823. doi:10.18653/v1/2022.findings-acl.142.

[57] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, J. Steinhardt, Measuring mathematical problem solving with the MATH dataset, in: Advances in Neural Information Processing Systems: Datasets and Benchmarks Track (NeurIPS), 2021.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Table 14
Auxiliary adaptive-replay mechanism and budget diagnostics. Panel A: Decomposition and model mix across tiers. Panel B: Target and realized token fractions under varying budget constraints.

Panel A: Mechanism Diagnostics and Model Mix

Method	Decomp.	QA Dec.	Math Dec.	RAM Act.	RAM Bias	Model Mix
GEAR full	13.0	13/50	0/50	0.0	0.0	61.7/5.1/33.2
GEAR static flat	0.0	0/50	0/50	0.0	0.0	34.5/32.5/33.0
GEAR raw memory	5.0	5/50	0/50	0.0	0.0	43.8/17.0/39.3
GEAR + RAM	12.0	12/50	0/50	100.0	0.0	46.8/21.2/32.0

Panel B: Adaptive-Replay Budget Sweep Across Constraints

Setting	Target (%)	EM (%)	Cost (USD)	Realized (%)	Final EMA (%)	λ Mean
full default	**16.0**	**91.0**	**0.435**	**11.6**	**9.5**	**0.006**
b05	5.0	96.0	0.367	11.4	10.8	0.063
b10	10.0	95.0	0.442	11.9	9.9	0.044
b25	25.0	93.0	0.542	15.2	10.7	0.027
b40	40.0	95.0	0.525	13.4	11.0	0.001

[58] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (1979) 65–70.
[59] Q. McNemar, Note on the sampling error of the difference between correlated proportions or percentages, Psychometrika 12 (1947) 153–157. doi:10.1007/BF02295996.

A. Auxiliary Adaptive-Replay Diagnostics

The remaining experiments examine stream-level adaptation dynamics under continuous online feedback, characterizing learning trajectories and fine-grained mechanism behaviors. Table 14 provides overarching mechanism and budget diagnostics across sweeps, while the following subsections detail the variant grid, topology density, and within-stream trajectories.

A.1. Short-stream variant grid

Table 15 reports the full variant grid on the 100-query adaptive replay, covering spatial, temporal, and value-gate ablations alongside both episodic-memory configurations.

A.2. Extended Diagnostics

Regarding fine-grained mechanism diagnostics, Panel A of Table 14 indicates that GEAR activates structural decomposition substantially more frequently on multi-hop question answering than on symbolic mathematics within this adaptive replay stream, reflecting empirical differences in sub-question modularity. Furthermore, memory-augmented configurations illustrate how conservative historical retrieval can modulate graph topology without destabilizing primary routing.

Regarding budget sensitivity, Table 14 (Panel B) and Fig. 5 trace empirical trajectories across targets from 5% to 40%. Realized token consumption remains tightly bounded within an 11.4%–15.2% regime, while task accuracy stays within a high-performance band of 91%–96% EM. Realized spend does not drop below 11.4% even under a 5.0% target, reflecting the irreducible resource floor of preliminary format-matched execution. Within this viable envelope, dynamic Lagrangian pacing modulates graph expansion effectively, preventing resource exhaustion while preserving reasoning fidelity.

Prefix tracking confirms rapid online stabilization: full GEAR achieves 90.0% EM across the initial 20 queries, 88.0% across 50, and converges to 91.0% over the complete stream (2.53 calls per query at $0.0048 per correct response).

Table 15
Adaptive-replay results on the 100-query stream. Every evaluated query updates the policy; costs are historical DashScope list-price estimates. "Math EM" and "QA EM" report domain-specific exact match; "Dec." counts episodes that trigger decomposition.

Method	EM (%)	Set-F1	Math EM	QA EM	Cost (USD)	Dec.
GEAR full	91.0	0.910	86.0	96.0	0.435	13
CoT (Qwen-Turbo)	82.0	0.771	84.0	80.0	0.012	0
CoT (Qwen-Plus)	84.0	0.829	88.0	80.0	0.102	0
CoT (Qwen-Max)	81.0	0.715	84.0	78.0	0.448	0
GEAR static flat	97.0	0.957	94.0	100.0	0.470	0
GEAR w/o spatial	90.0	0.921	88.0	92.0	0.373	0
GEAR w/o temporal	96.0	0.968	94.0	98.0	0.518	4
GEAR w/o value gate	95.0	0.958	92.0	98.0	0.465	15
GEAR w/ raw memory	93.0	0.951	92.0	94.0	0.487	5
GEAR w/ conservative RAM	93.0	0.921	90.0	96.0	0.496	12

Table 16
Execution-topology density. Density combines tier-weighted calls, decomposition sub-nodes, and extra refinement rounds. "Dens./Correct" is total density divided by correct answers (lower is more topology-efficient).

Method	Density	Calls/Q	Nodes/Q	EM (%)	Dens./Correct
GEAR full	**4.78**	**2.53**	**0.39**	**91.0**	**5.25**
CoT (Qwen-Plus)	2.00	1.00	0.00	84.0	2.38
CoT (Qwen-Max)	3.00	1.00	0.00	81.0	3.70
Static cascade	1.15	1.06	0.00	86.0	1.34
GEAR static flat	4.09	2.03	0.00	97.0	4.22
Always orchestrate	8.77	3.79	1.40	92.0	9.53
GEAR w/o value gate	5.86	2.64	0.57	95.0	6.17
GEAR raw memory	4.64	2.24	0.18	93.0	4.99
GEAR + RAM	5.07	2.50	0.39	93.0	5.45

A.3. Execution-Topology Density Diagnostics

To evaluate orchestration complexity, each replayed execution trace is scored via a composite density metric combining tier-weighted LLM calls (Turbo = 1, Plus = 2,

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005

Max = 3), decomposition sub-nodes, and iterative refinement rounds. As shown in Table 16, GEAR attains 91.0% EM at a structural density of 4.78 (5.25 units per correct response). Unconstrained Always-orchestrate incurs an inflated density of 8.77 (9.53 units per correct response) for 92.0% EM, confirming that GEAR halves structural overhead while preserving reasoning fidelity. Restricting intermediate sub-node creation to difficult compositional queries prevents topology inflation on straightforward instances. Ablating the value gate raises density to 5.86 without accuracy gain, demonstrating that value gating halts uncontrolled graph expansion.

A.4. Adaptive-Replay Dynamics under Continuous Workload Streams

To assess adaptation under continuous feedback, we evaluate a sequential 300-query mixed stream with online updates. GEAR attains 91.3% EM at $0.430 per query and 1.86 provider calls, outperforming static flat orchestration (89.7% EM, $1.299, 2.16 calls). Baseline CoT (Qwen-Plus) scores 80.3% at $0.237, while unconstrained episodic memory reaches 84.7% at $1.461. These empirical trajectories demonstrate the rapid convergence and cost-effectiveness of our task-governed architecture. By updating posterior beliefs without abrupt parameter shifts, the Bayesian router preserves stability across heterogeneous workloads while preventing runaway context inflation.

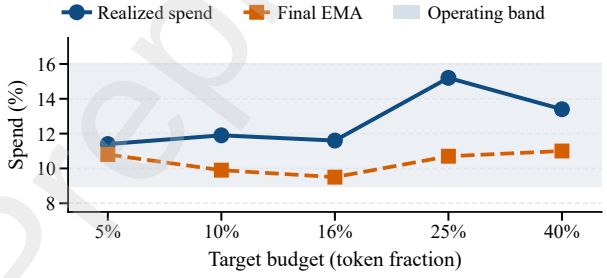


Figure 5: Budget-pacing behavior in the adaptive-replay sweep (Table 14, Panel B). Realized token fractions saturate in a narrow operating band, while exact match stays between 91% and 96%.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=7541005